



中国科学院大学

University of Chinese Academy of Sciences

硕士学位论文

基于 LoongArch 架构的 ART 执行引擎 适配与优化技术研究

作者姓名: 鄢玺

指导教师: 张福新 正高级工程师

中国科学院计算技术研究所

学位类别: 工学硕士

学科专业: 计算机科学与技术

培养单位: 中国科学院计算技术研究所

2026 年 6 月

Research on Adaptation and Optimization of
the ART Execution Engine
for LoongArch Architecture

A thesis submitted to
University of Chinese Academy of Sciences
in partial fulfillment of the requirement
for the degree of
Master of Engineering
in Computer Science and Technology

By

Yan Xi

Supervisor: Senior Engineer Zhang Fuxin

Institute of Computing Technology, Chinese Academy of Sciences

June, 2026

中国科学院大学 学位论文原创性声明

本人郑重声明：所呈交的学位论文是本人在导师的指导下独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明或致谢。

作者签名：

日 期：

中国科学院大学 学位论文授权使用声明

本人完全了解并同意遵守中国科学院有关保存和使用学位论文的规定，即中国科学院有权保留送交学位论文的副本，允许该论文被查阅，可以按照学术研究公开原则和保护知识产权的原则公布该论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存、汇编本学位论文。

涉密及延迟公开的学位论文在解密或延迟期后适用本声明。

作者签名：

日 期：

导师签名：

日 期：

摘 要

面向国产处理器平台完善 Android 应用执行环境，是推进基础软件生态建设的重要环节。AOSP 15 官方主线尚未提供面向 LoongArch 架构的 ART 执行引擎实现，这使得 Android 应用在该平台上的稳定运行与性能优化缺少关键运行时基础。本文以 AOSP 15 的 Android Runtime (ART) 为研究对象，围绕 LoongArch 平台执行引擎的适配设计、正确性验证与性能优化展开系统研究。

本文首先面向 LoongArch 平台完成了 ART 执行引擎关键模块的适配，系统补齐了汇编器、C++ 解释器、Nterp、JIT 编译器后端、JNI 编译器、运行时快速路径以及部分 Intrinsic 支持，构建了 LoongArch 平台上可运行的 ART 执行环境。其次，本文建立了覆盖编译、部署、目标机回归、libcore 测试与基准测试评测的统一验证链路，形成了固定 JIT 基线、日志归档与热点分析方法，为移植结果和后续优化实验提供了可复现支撑。

在性能优化方面，本文基于统一基线对典型工作负载的热点行为进行分析，围绕 DaCapo/pmd、SPECjvm2008/mpegaudio 与 Scimark 数值子项提出并验证了三项正式优化方案：JIT 分层热度阈值调整方案、运行时字符串搬运批量化方案，以及 LoongArch64 LSX 浮点 SIMD 向量化方案。实验结果表明，JIT 分层热度阈值调整方案在 pmc 上存在可复现的正收益样本，但重复测量仍显示较大方差；运行时字符串搬运批量化方案仅呈现弱正收益，更适合作为字符串密集场景的定向微优化；LoongArch64 LSX 浮点 SIMD 向量化方案则在 SPECjvm2008/scimark.lu.small 上取得了稳定且显著的性能提升，是当前证据最完整的主收益结论。进一步结合方案叠加负例与若干失败方案的对照分析，本文归纳出 LoongArch 平台 ART 执行引擎优化应优先命中主导热点、严格控制全局副作用，并明确不同工作负载的收益边界。

研究表明，AOSP 15 ART 在 LoongArch 平台上的执行引擎适配是可行的；在统一验证链路和固定基线约束下，围绕执行引擎热点开展定向优化能够获得可复现收益。本文工作为国产处理器平台上 Android 运行时的进一步完善与性能提升提供了工程基础和方法参考。

关键词： LoongArch，Android Runtime，执行引擎适配，JIT 编译，性能优化

Abstract

Improving the Android application runtime environment on domestic processor platforms is an important part of building a complete system software ecosystem. However, AOSP 15 does not provide an official ART execution engine implementation for the LoongArch architecture, which leaves a critical gap in application execution support on this platform. This thesis takes Android Runtime (ART) in AOSP 15 as the research target and studies execution engine adaptation, correctness validation, and performance optimization for LoongArch.

First, this work completes the adaptation of key ART execution engine components for LoongArch, including the assembler, C++ interpreter, Nterp, the JIT compiler backend, the JNI compiler, runtime fast paths, and selected intrinsics, thereby establishing a runnable ART execution environment on the target platform. Second, it builds a unified validation workflow covering compilation, deployment, target-side regression tests, libcore tests, and benchmark evaluation, and further establishes a fixed JIT baseline together with reproducible logging and hotspot analysis procedures.

For performance optimization, this thesis analyzes hotspot behavior under a fixed baseline and studies three formal schemes on representative workloads: a tiered JIT hotness threshold adjustment, a runtime string bulk-copy optimization, and a LoongArch64 LSX floating-point SIMD vectorization scheme. The results show that the tiered JIT threshold adjustment has reproducible positive samples on DaCapo/pmd but still exhibits notable variance under repeated measurements; the runtime string bulk-copy optimization only delivers weak gains and is therefore better characterized as a targeted micro-optimization for string-intensive workloads; and the LoongArch64 LSX floating-point SIMD scheme achieves a stable and substantial improvement on scimark.lu.small, making it the strongest positive result supported by the current evidence. By further comparing stacking conflicts and several negative results, this thesis concludes that optimization of the ART execution engine on LoongArch should prioritize dominant hotspots, strictly control global side effects, and explicitly state workload applicability

boundaries.

The results demonstrate that adapting the AOSP 15 ART execution engine to the LoongArch platform is feasible, and that targeted hotspot-driven optimizations under a unified validation workflow and a fixed baseline can produce reproducible benefits. This work provides both an engineering foundation and a practical methodology reference for further improving Android runtime support on domestic processor platforms.

Keywords: LoongArch, Android Runtime, execution engine adaptation, JIT compilation, performance optimization

目 录

第 1 章 绪论	1
1.1 研究背景与意义	1
1.1.1 研究背景	1
1.1.2 研究意义	1
1.2 国内外研究现状	2
1.2.1 Android 运行时与混合编译研究	2
1.2.2 解释器与执行引擎优化研究	2
1.2.3 LoongArch 平台 JVM 与 ART 移植研究	3
1.3 本文研究内容与主要贡献	3
1.3.1 研究问题	3
1.3.2 研究内容	4
1.3.3 主要贡献	4
1.4 论文组织结构	5
第 2 章 相关技术与问题分析	7
2.1 ART 运行时体系结构	7
2.2 ART 的执行与编译机制	7
2.2.1 DEX 字节码与寄存器虚拟机	7
2.2.2 C++ 解释器、Nterp 与解释器优化机制	8
2.2.3 JIT、AOT 与分层编译机制	9
2.2.4 Intrinsic 与运行时快速路径	9
2.3 LoongArch 架构特性及其对 ART 的影响	10
2.3.1 寄存器体系与调用约定	10
2.3.2 控制转移与地址计算特点	11
2.3.3 内存访问、原子操作与同步语义	11
2.4 性能评测与优化问题分析	12
2.4.1 经典 JVM benchmark 的代表性	12
2.4.2 Warmup、波动与统计严谨性	12
2.4.3 低扰动可观测性需求	12
2.5 AOSP 15 ART 移植的关键技术难点	13
2.5.1 执行引擎适配范围广且耦合紧密	13
2.5.2 正确性验证与性能研究必须并行考虑	13
2.5.3 新版本 ART 的复杂性与历史可比性问题	14
2.6 本章小结	14

第 3 章 LoongArch 平台 ART 执行引擎的适配设计与实现	15
3.1 总体设计目标与实现策略	15
3.2 汇编器与基础代码生成支持	16
3.2.1 LoongArch64 汇编器实现	16
3.2.2 分支、字面量与链接修补机制	17
3.3 解释器适配	18
3.3.1 C++ 解释器适配	18
3.3.2 Nterp 框架建立	18
3.3.3 Nterp 指令补齐与问题修复	19
3.4 JIT 编译器后端适配	20
3.4.1 LoongArch64 代码生成器实现	20
3.4.2 核心 HIR 节点支持	20
3.4.3 OSR 与运行时协同支持	21
3.5 JNI 编译器与运行时快速路径实现	21
3.5.1 JNI 调用桥接实现	21
3.5.2 运行时入口点与快速路径补齐	22
3.6 Intrinsic 与执行引擎能力扩展	22
3.7 阶段性结果分析	23
3.8 本章小结	23
第 4 章 系统验证方法与实验平台	25
4.1 实验环境与软硬件平台	25
4.2 编译、部署与测试流程	25
4.2.1 编译流程	25
4.2.2 部署流程	26
4.2.3 验证链路组织方式	26
4.3 正确性验证方法	27
4.3.1 宿主机侧测试	27
4.3.2 目标机侧 run-test 验证	27
4.3.3 libcore 测试	28
4.3.4 差分测试与辅助调试方法	28
4.4 基准测试设计与正式基线建立	29
4.4.1 基准测试选择原则	29
4.4.2 正式基线口径	30
4.4.3 性能数据采集方式	30
4.5 有效性威胁与实验控制	31

4.6 验证结果汇总	32
4.6.1 移植阶段验证结果	32
4.6.2 优化阶段验证要求	32
4.7 本章小结	32
第 5 章 面向 LoongArch 平台 ART 的性能优化研究	35
5.1 优化目标与分析方法	35
5.2 基线工作负载特征分析	37
5.2.1 DaCapo pmd 特征	37
5.2.2 SPECjvm2008 mpegaudio 特征	38
5.3 JIT 分层热度阈值调整（方案一）	39
5.3.1 问题背景与设计思路	39
5.3.2 实现方法	40
5.3.3 实验结果与分析	41
5.4 运行时字符串搬运批量化（方案二）	42
5.4.1 问题背景与设计思路	42
5.4.2 实现方法	42
5.4.3 实验结果与分析	43
5.5 LoongArch64 LSX 浮点 SIMD 向量化（方案三）	44
5.5.1 问题背景与设计思路	44
5.5.2 实现方法	44
5.5.3 实验结果与分析	45
5.5.4 完整 SIMD 覆盖版的后续消融	46
5.6 三项正式方案的统一对比与重复测量验证	47
5.6.1 同日正式单次结果总览	47
5.6.2 同日重复测量验证	48
5.6.3 三项方案的叠加并不天然成立	49
5.7 Intrinsic 机制的补充验证	49
5.8 失败方案对照分析	52
5.8.1 解释器与 JIT 热路径类失败方案	52
5.8.2 运行时微优化类失败方案	53
5.8.3 失败方案的共性原因	54
5.9 正收益方案与失败方案的对比总结	57
5.9.1 可门控的局部调整与最小能力补齐更易兑现收益	57
5.9.2 不同 workload 的收益边界	57
5.9.3 面向 ART 执行引擎的优化方法论	57
5.10 本章小结	58

第 6 章 总结与展望	59
6.1 本文工作总结	59
6.2 主要工作与贡献凝练	59
6.2.1 完成 LoongArch 平台 ART 执行引擎适配	59
6.2.2 建立系统化验证与实验方法	60
6.2.3 获得面向典型 workload 的优化结论	60
6.2.4 形成以失败样本为约束的优化研究方法	60
6.3 工作局限性分析	61
6.3.1 benchmark 覆盖范围仍有限	61
6.3.2 部分结论受 debug 构建口径约束	61
6.3.3 部分候选方向仍需进一步探索	61
6.3.4 因果证据仍以外部观测为主	62
6.4 后续工作展望	62
6.4.1 完善 AOT 与安装期编译支持	62
6.4.2 探索 LoongArch 向量扩展优化	62
6.4.3 扩展 benchmark 与应用级验证	63
6.4.4 增强低扰动观测与运行时因果分析	63
6.5 本章小结	63
附录 A 补充实验：DexFileVerifier 阶段链实验	65
参考文献	67
致谢	71
作者简历及攻读学位期间发表的学术论文与研究成果	73

图形列表

5.1 完整 SIMD 覆盖候选版在关键 workload 上的相对收益	46
5.2 三项正式方案相对同日基线的归一化收益热力图	48
5.3 同日重复测量下方案一与方案三的原始散点和均值	48
5.4 LoongArch64 Intrinsic 开关消融实验的几何平均 speedup	50
5.5 LoongArch64 Intrinsic 开关消融实验的逐 workload 收益热力图	51
5.6 已归档失败方案相对固定 baseline 的归一化收益热力图	55
A.1 DexFileVerifier 阶段链实验的归一化收益	65

表格列表

3.1 LoongArch 平台 ART 执行引擎适配的阶段化策略	16
4.1 目标机侧 run-test 验证结果	28
5.1 本文主要实验版本的语义化命名	36
5.2 DaCapo/pmd baseline 主要热点符号	38
5.3 SPECjvm2008/mpegaudio baseline 主要热点符号	38
5.4 pmd 与 mpegaudio 的特征对比	39
5.5 JIT 分层热度阈值调整方案结果	41
5.6 运行时字符串搬运批量化方案结果	43
5.7 LoongArch64 LSX 浮点 SIMD 向量化方案结果	45
5.8 完整 SIMD 覆盖候选版的几何平均 speedup	46
5.9 方案叠加状态与单方案结果对照	49
5.10 Intrinsic 开关消融实验的几何平均 speedup	50
5.11 与 JIT 热路径相关的失败方案对照	52
5.12 与运行时搬运相关的失败方案对照	53

符号列表

缩略语

缩略语	含义
AOSP	Android Open Source Project
ART	Android Runtime
AOT	Ahead-Of-Time compilation
JIT	Just-In-Time compilation
JNI	Java Native Interface
DEX	Dalvik Executable bytecode format
OSR	On-Stack Replacement
IR	Intermediate Representation
ISA	Instruction Set Architecture
LSX	Loongson SIMD eXtension
LASX	Loongson Advanced SIMD eXtension

实验记号

记号	含义
msec	DaCapo 类 benchmark 的执行耗时指标，数值越小越好
ops/m	SPECjvm2008 类 benchmark 的吞吐指标，数值越大越好
baseline	本文统一采用的固定对照版本与实验口径
workload	用于性能分析与验证的具体 benchmark 子项或应用负载

第 1 章 绪论

1.1 研究背景与意义

1.1.1 研究背景

近年来，国产处理器与基础软件生态建设持续推进，LoongArch 作为面向自主可控需求设计的指令集架构，在体系结构设计、生态兼容和扩展能力方面不断完善 [1–3]。与此同时，移动操作系统与应用生态已成为处理器软件支持能力的重要体现。Android 长期占据全球移动操作系统主导地位 [4]，其运行时系统直接影响应用启动速度、交互时延和整体执行效率。因此，在国产处理器平台上完善 Android 核心运行环境，不仅是单纯的代码移植工作，更涉及生态可用性与性能可接受性的系统性挑战。

Android Runtime (ART) 是 Android 应用执行的核心运行时。ART 承担 DEX 字节码解释执行、即时编译、安装期预编译、JNI 调用桥接、类加载和运行时支持等任务 [5–7]。相较于早期 Dalvik，ART 采用解释执行、Just-In-Time (JIT) 即时编译与 Ahead-Of-Time (AOT) 预编译相结合的混合执行模式，在启动开销、生成代码质量与运行时资源占用之间寻求平衡 [8–11]。这一机制决定了 ART 的平台适配并不是单点模块移植，而是涉及解释器、编译器后端、JNI 编译器和运行时入口点等多个子系统的系统性工程。

然而，AOSP 15 官方主线并未提供面向 LoongArch 的 ART 执行引擎实现。对于新架构平台而言，即使 Linux 内核、Bionic 和系统框架层已经具备基础可运行能力，若缺少 ART 层支持，Android 应用生态仍难以真正落地。特别是 ART 中与体系结构密切相关的解释器入口、Nterp 汇编解释器、JIT 代码生成、JNI 调用约定和快速路径入口点等模块，均需要针对目标架构进行专门适配。此外，在完成“能运行”的功能目标之后，仍需进一步解决“能正确”和“能更快”的问题，即通过系统回归验证保证语义正确性，并通过性能分析与优化降低新平台上的执行开销。

1.1.2 研究意义

围绕 LoongArch 平台 ART 执行引擎的适配与优化开展研究，具有以下几个方面的重要意义。

第一，本文工作有助于补齐国产处理器上的 Android 运行时关键能力。相比通用用户态程序移植，运行时系统直接决定 Java 与 Kotlin 应用是否能够稳定、高效地在目标平台执行，因此 ART 的移植结果对移动生态可用性具有基础性影响。

第二，本文工作能够积累新架构平台运行时系统适配的方法与经验。ART 并非单纯的解释器或编译器，而是一个由解释执行、JIT/AOT 编译、JNI 桥接、异常处理、调试信息和运行时快路径共同构成的复杂系统。对其进行平台化适配，有助于形成面向新架构的执行引擎移植方法论。

第三，本文工作为 LoongArch 平台上的性能优化研究提供了可复现基础。只有在执行引擎基础能力补齐并通过系统验证后，围绕特定 workload 的性能瓶颈分析与优化才具有现实意义。本文在完成移植适配的基础上进一步开展 benchmark 驱动的优化研究，既能给出有效方案，也能通过失败方案对照总结出更贴近 ART 实际行为的优化规律。

1.2 国内外研究现状

1.2.1 Android 运行时与混合编译研究

Android 运行时经历了从 Dalvik 向 ART 的演进过程。随着应用规模扩大，仅依赖解释执行已难以满足性能需求，运行时中逐步引入 JIT 与 AOT 混合编译机制 [8, 9, 12]。相关研究表明，移动端运行时的关键在于根据热点形成节奏与编译成本之间的权衡设计合理的混合策略，而非单纯追求更激进的编译等级 [10, 11, 13]。

围绕 Android 平台 JIT 行为本身，也已有工作从参数、策略和代码缓存等角度开展优化研究。例如，配置感知型运行时优化工作关注不同负载与配置条件下的 JIT 行为差异，强调运行时参数对最终性能的影响 [14]；面向应用启动加速的工作则尝试通过更具针对性的 JIT 策略缩短关键路径上的解释执行与编译等待时间 [15]。这些研究说明，ART 的性能优化通常需要紧密结合 workload 生命周期和热点路径特征，不能简单依赖单一静态阈值或全局默认策略。

1.2.2 解释器与执行引擎优化研究

在执行引擎层面，解释器性能长期是虚拟机研究中的重要问题。经典研究指出，解释器的分派结构与分支预测行为会显著影响整体吞吐 [16]；与此同时，JIT 编译虽能提升代码质量，但也会引入编译线程开销与调试信息等固定成本

[17–19]。因此，执行引擎优化的关键在于平衡解释器驻留时间与 JIT 编译触发时机，而非单纯追求更早或更激进的编译行为。关于解释器分派机制和 JIT 分层策略的详细技术分析，将在第 2 章中进一步展开。

对于字符串与运行时热路径优化，除解释器与编译器层面的优化外，运行时热路径中的具体操作同样值得关注。已有研究表明，Java 字符串对象的存储布局、复制方式和字符编码转换路径会对内存占用与执行效率产生持续影响。特别是在大量短字符串构造、拼接与替换场景中，热路径中的逐元素搬运和额外控制开销容易成为稳定瓶颈。因此，围绕字符串处理路径开展局部、低副作用的微优化，仍然是运行时系统中具有现实价值的方向。

1.2.3 LoongArch 平台 JVM 与 ART 移植研究

在国产处理器平台上的虚拟机移植方面，已有研究围绕 HotSpot、Dalvik 和早期 ART 版本展开。黄佳的工作完成了基于 LoongArch64 指令集的 Java 虚拟机移植与优化，验证了在新架构平台上补齐汇编器、解释器和编译器后端的可行性[20]。面向 Android 运行时的已有研究则更多聚焦于 MIPS 平台或较早期 Android 版本上的运行时适配[21]。这些工作为本文提供了方法和经验上的参考，但也存在两个不足：其一，研究对象多为早期版本虚拟机，未覆盖 AOSP 15 中更复杂的 ART 执行引擎形态；其二，已有工作通常更偏向“完成移植”，而缺少围绕固定 baseline 的系统化优化与失败方案对照分析。

因此，面向 AOSP 15 的 LoongArch 平台 ART 执行引擎适配与优化仍然具有明显研究空间。尤其是在新版本 ART 已同时包含 C++ 解释器、Nterp、JIT/AOT 编译、JNI 编译和更复杂运行时支撑机制的背景下，如何完成系统适配，并在此基础上开展更有针对性的优化研究，是一个兼具工程价值与研究价值的问题。

1.3 本文研究内容与主要贡献

1.3.1 研究问题

本文不将 LoongArch 平台上的 ART 研究视为简单的文件补齐与测试通过工作，而是将其收束为三个相互关联的研究问题。

第一，在 AOSP 15 这一较新版本 ART 上，LoongArch 平台执行引擎的最小可验证闭环究竟由哪些关键模块构成，应当按照怎样的实现顺序逐步建立。这一问题对应第 3 章的适配设计与实现，核心是明确解释器、Nterp、JIT 编译器后

端、JNI 编译器和运行时快速路径之间的依赖关系，而不是将平台适配理解为若干离散补丁的简单累加。

第二，在执行引擎适配尚处于收敛阶段的前提下，如何建立一条既能约束语义正确性、又能支撑后续性能分析的统一验证链路。这一问题对应第 4 章，核心是把编译、部署、target 回归、libcore 测试与 benchmark 评测组织成固定流程，并通过统一 baseline 和统一归档口径保证实验结论可复核。

第三，在固定 baseline 与统一验证约束下，LoongArch 平台 ART 当前真正值得保留的优化方向是什么，不同方案的收益强度和适用边界又应如何界定。这一问题对应第 5 章，核心不是追求更多跑分提升，而是区分稳定收益、边界性收益与失败样本，形成对执行引擎优化更可信的筛选结论。

1.3.2 研究内容

本文以 AOSP 15 的 ART 为研究对象，围绕 LoongArch 平台执行引擎的适配与优化开展工作。整体研究内容可以概括为以下三个方面。

第一，完成 LoongArch 平台 ART 执行引擎的适配设计与实现。本文以执行引擎相关模块为重点，系统补齐汇编器、C++ 解释器、Nterp 汇编解释器、JIT 编译器后端、JNI 编译器和运行时快速路径等核心能力，使 ART 在 LoongArch 平台上具备完整执行基础。

第二，建立面向目标设备的系统验证与实验框架。本文围绕编译、部署、target 回归、libcore 验证和 benchmark 评测建立统一流程，确保移植结果在语义正确性和可复现性两个层面都得到支撑，并构建了后续性能优化所依赖的正式 baseline。

第三，围绕固定 baseline 开展执行引擎优化研究。本文结合典型 benchmark workload 的行为特征，重点分析解释器驻留、JIT 触发、字符串热路径和浮点数值内核等问题，形成三项正式优化方案，并通过重复测量验证、叠加负例和失败方案对照，区分不同方案的证据强度、适用边界与方法论含义。

1.3.3 主要贡献

本文的主要贡献如下。需要说明的是，这些贡献并非处于完全相同的证据层级：执行引擎适配闭环与统一验证链路属于全文的稳定基础结论；三项正式优化方案属于在统一口径下筛选得到的正式研究对象，其中方案三的收益证据当前最为完整，方案一和方案二的稳定性结论仍将在后续控制实验中继续补强。

1. 面向 AOSP 15 完成了 LoongArch 平台 ART 执行引擎关键模块的适配，覆盖解释器、Nterp、JIT 编译器后端、JNI 桥接和运行时快速路径等核心子系统，并给出了从最小可运行闭环到主要执行路径全部打通的实现组织方式。

2. 建立了覆盖编译、部署、target 回归、libcore 与 benchmark 的统一验证链路，为移植结果和优化实验提供了可复现的评测基础，并通过固定 baseline、日志归档和热点分析约束了后续性能结论的解释边界。

3. 基于固定 baseline 提出了三项正式优化方案，分别针对 JIT 分层热度阈值、运行时字符串热路径和 LoongArch64 LSX 浮点 SIMD lowering 开展优化，并在统一口径下完成了收益验证与边界分析。其中，方案三在当前材料中形成了最稳定、最完整的正收益证据，方案一和方案二则展示了执行引擎优化在不同 workload 上的收益差异与后续验证重点。

4. 通过同日重复测量、叠加负例与失败方案对照，总结了面向 LoongArch 平台 ART 执行引擎优化的实践经验，涵盖热点预算评估、全局副作用控制、收益稳定性判断以及不同 workload 的适用边界等方面。

1.4 论文组织结构

全文共分为六章，各章内容安排如下。

1. 第 1 章为绪论，介绍本文的研究背景、研究意义、国内外研究现状、研究内容、主要贡献以及论文整体结构。

2. 第 2 章为相关技术与问题分析，主要介绍 ART 运行时体系结构、执行与编译机制、LoongArch 架构特性以及 AOSP 15 ART 移植面临的关键技术难点。

3. 第 3 章为 LoongArch 平台 ART 执行引擎的适配设计与实现，重点阐述汇编器、解释器、Nterp、JIT 编译器后端、JNI 编译器和运行时快速路径等模块的实现过程。

4. 第 4 章为系统验证方法与实验平台，介绍实验环境、编译部署链路、正确性验证方法以及正式 baseline 的建立方式。

5. 第 5 章为面向 LoongArch 平台 ART 的性能优化研究，围绕三项正式方案、同日重复测量验证、叠加负例以及失败方案对照展开分析，给出不同证据强度下的收益边界与方法论总结。

6. 第 6 章为总结与展望，对全文工作进行归纳，并给出后续在 AOT 支持、

更完整的向量化优化和 benchmark 扩展等方面的进一步研究方向。

第 2 章 相关技术与问题分析

2.1 ART 运行时体系结构

Android Runtime (ART) 是 Android 平台上负责 Java/Kotlin 应用执行的核心运行时, 其职责并不局限于“解释执行字节码”, 而是贯穿应用装载、字节码校验、解释执行、即时编译、安装期编译、垃圾回收、JNI 桥接与异常处理等完整链路 [5, 7]。从源码组织上看, runtime/ 负责线程、对象模型、类链接、垃圾回收与运行时服务, interpreter/ 负责解释执行路径, compiler/ 与 dex2oat/ 负责编译基础设施, libdexfile/ 负责 DEX 文件解析与校验, 多个子模块共同构成一个分层协同的执行系统。

与传统“单一解释器”式虚拟机不同, ART 的一个显著特点是执行模式并存。应用方法在不同阶段可能经过 DEX 校验、解释执行、baseline JIT 编译、优化重编译以及安装期 AOT 编译等多条路径; 运行时则负责在这些路径之间维护统一的对象语义、异常语义和线程状态 [6, 17]。因此, 对 ART 的平台适配不能简单理解为某个独立模块的移植, 而必须把解释器、编译器、运行时入口点、调用约定和调试支撑作为一个整体来看待。

从 Android 应用的实际执行链路看, ART 既要保证冷启动阶段的可用性, 又要保证热点路径上的执行效率。这种双重目标决定了 ART 的工程组织方式天然具有层次性: 以解释器和运行时为最小可运行闭环, 以 JIT 和 AOT 作为进一步提升吞吐和启动性能的高性能路径, 以 Intrinsic、快速入口点和 JNI 桥接作为连接 Java 语义与机器级实现的关键补充。对本文而言, 理解这种层次结构, 是分析 LoongArch 平台适配范围与后续优化边界的前提。

2.2 ART 的执行与编译机制

2.2.1 DEX 字节码与寄存器虚拟机

ART 的上层执行对象是 DEX 字节码。与传统 JVM 栈式字节码不同, DEX 采用寄存器虚拟机模型, 字节码指令通常显式给出源寄存器与目标寄存器, 从而减少中间入栈、出栈操作, 提高指令表达密度和移动端执行效率 [8, 12]。这种设计有利于降低解释器中的栈操作频率, 也为后续的 SSA 构图、寄存器分配和机

器码生成提供了更直接的中间表示基础。

寄存器虚拟机模型并不意味着平台适配工作更简单。相反，它要求解释器、JIT 编译器和运行时在寄存器槽位、ShadowFrame、方法调用参数区以及返回值位置之间保持一致映射。一旦平台相关调用约定、对象访问语义或异常恢复路径处理不当，DEX 层面看似简单的字段读写、类型检查和方法调用就会在目标机上表现为寄存器污染、栈帧破坏或错误异常传播。因此，DEX 的寄存器化设计一方面提高了执行潜力，另一方面也要求平台适配必须在执行语义和底层调用约定之间建立严格对应关系。

2.2.2 C++ 解释器、Nterp 与解释器优化机制

解释器是 ART 最基础、也是最具可移植性的执行路径。C++ 解释器依赖平台无关的 switch-case 主循环实现字节码分派，便于在新架构上较早建立“可执行、可调试、可验证”的最小闭环。然而，从解释器性能研究的经典结论可知，解释执行的瓶颈往往不仅来自单条指令实现本身，也来自分派结构和间接分支预测行为 [16]。这也是为什么 ART 在 C++ 解释器之外，又引入了 Nterp 这类面向热点解释执行的汇编解释器路径。

Nterp 的核心思想是通过预构建派发表和固定寄存器协议，将 opcode 直接映射到对应的汇编处理块，从而减少传统 switch-case 分派中的共享分支开销。其设计与高效解释器研究中的 threaded code 思路是一致的：通过让不同指令拥有更明确的控制流上下文，提高处理器对分派路径的可预测性 [16]。在此基础上，quickenning、inline cache 等解释器优化技术则进一步说明，解释器的优化空间并不只在分派层面，还可以体现在“把常见慢路径提前特化”为更便宜的执行形态 [22, 23]。

对本文的研究对象而言，这些结论具有直接启发意义。LoongArch 平台 ART 的适配不应仅以“能否运行解释器”为终点，而应把解释器路径视为整个执行引擎的语义基线与性能基线。只有先建立与 C++ 解释器一致的语义，再逐步补齐 Nterp 的入口、帧布局、寄存器约束和字节码处理逻辑，后续 JIT、OSR 和运行时协同路径才有稳定的参照系。

2.2.3 JIT、AOT 与分层编译机制

在解释执行之上，ART 通过 JIT 与 AOT 形成混合执行机制。AOT 通常在安装期或系统镜像构建阶段完成，以降低启动时的即时编译成本；JIT 则在运行期根据方法热度动态生成本地代码，以在不牺牲灵活性的前提下提升热点路径性能 [6, 9]。从编译系统角度看，这属于典型的自适应优化框架：运行时先以较低成本收集热点信息，再将编译资源集中投向最有价值的方法 [11, 17]。

ART 当前的 JIT 机制并非单级编译即终止的模式，而是具有明显的分层特征。方法首先需要达到 warmup 阈值进入 baseline JIT，在热点继续累积后，再可能触发更高质量的优化重编译；若热点循环在解释执行阶段已经形成，还可以通过 On-Stack Replacement (OSR) 从解释器切换到已编译代码 [18, 19]。这种层次化策略的好处在于兼顾了编译时机和机器码质量，但它也引入了新的平台相关问题：阈值如何设定、代码缓存如何组织、调试信息与编译固定成本如何控制，都会直接影响不同 workload 的最终收益边界。

移动端 JIT 研究进一步表明，“何时编译、编译到什么程度、是否让不同编译策略协同”本身就是性能优化的重要部分，而不是编译器后端之外的边角参数 [14, 15, 24]。因此，在 LoongArch 平台上研究 ART 性能优化时，不能把 JIT 参数调整简单视为工程调参，而应当视为分层执行机制与具体 workload 生命周期之间的匹配问题。

2.2.4 Intrinsic 与运行时快速路径

仅有解释器与编译器主体还不足以支撑成熟运行时。ART 中大量高频 Java 标准库方法依赖 Intrinsic 实现，以便在编译阶段直接映射为更短的机器码序列或调用特定运行时辅助函数；与此同时，对象分配、字段访问、类型检查、字符串构造、异常抛出和 JNI 调用等路径，也普遍依赖 art_quick_* 入口点或相关 trampoline 提供快速服务 [5, 6]。

从执行引擎角度看，Intrinsic 与快速路径承担着“缩短热点语义到机器实现距离”的角色。一方面，它们减少了通用调用路径上的层层间接开销；另一方面，它们也是不同架构平台体现特化能力的重要位置。例如，字符串相关运行时路径的实现方式会直接影响短字符串密集 workload 的热点分布，而编译器是否能为标准库方法生成 Intrinsic 代码，则会决定新架构平台在成熟度和性能上的上

限 [25]。

需要区分的是，**Intrinsic** 与普通方法内联并不等价。普通内联仍然以被调方法的中间表示为输入，通过复制或重写 **HIR** 后再交给后续优化阶段处理；**Intrinsic** 则更像一条预先声明的语义捷径：编译器在识别到特定标准库方法后，可直接为其建立专用的 **locations** 和 **lowering**，并发射更短的机器码序列；若当前架构尚未补齐对应实现，则自动回退到普通 **invoke** 路径。因此，**Intrinsic** 的平台适配并不是“多做几条 **peephole** 优化”，而是在执行引擎中为一组可枚举的库语义建立离散映射表。

从优化对象看，**Intrinsic** 与循环 **SIMD** 向量化也分属两个层次。前者主要面向 **Math**、**String**、数组辅助方法以及部分加解密等高频库调用，收益通常体现为缩短调用边界和减少固定指令序列；后者则主要面向编译器能够识别的显式循环与向量数据流。二者都属于“缩短高层语义到机器实现距离”的手段，但命中的热点形态并不相同：**Intrinsic** 更偏向库调用和内建语义路径，循环 **SIMD** 则更偏向显式规则数值循环和可识别的数据并行模式。

因此，对新架构平台的 ART 适配而言，补齐 **Intrinsic** 与运行时快速路径并非可选的附加优化，而是让高层 **Java** 语义真正落到可接受机器级实现上的必要条件。这也是为什么本文后续在移植阶段将其归入执行引擎能力补齐的组成部分，而在优化阶段又围绕其中若干热点路径继续展开性能研究。

2.3 LoongArch 架构特性及其对 ART 的影响

2.3.1 寄存器体系与调用约定

LoongArch 是面向现代通用处理器设计的 **RISC** 指令集，采用定长 32 位指令格式，并提供独立的通用寄存器与浮点寄存器集合 [1, 3]。从 ART 适配视角看，最直接的影响来自平台调用约定：参数如何在寄存器与栈之间传递，返回值如何编码，多返回类型如何区分，调用者保存与被调用者保存寄存器如何划分。这些规则会同时作用于解释器入口、JIT 代码生成、JNI 桥接和运行时慢路径恢复。

与平台无关的中间表示相比，执行引擎真正落地到目标机时，调用约定是最先暴露问题、也是最难绕开的约束。例如，**ShadowFrame** 与 **native** 栈之间如何衔接，**Thread***、**ArtMethod*** 和参数区在入口时如何布局，浮点返回值是否需要

特殊寄存器通道，这些问题都无法通过“平台无关逻辑”自动解决。因此，在 LoongArch 平台上适配 ART，必须首先建立清晰、稳定且与架构 ABI 一致的寄存器与调用约定映射关系。

2.3.2 控制转移与地址计算特点

LoongArch 的另一项重要特性，是其控制转移与地址构造方式对代码生成组织有较强影响。由于指令长度固定，分支位移范围、长短跳转组合、PC 相对寻址以及字面量地址构造都需要在汇编器和代码生成器中显式处理 [3]。这对 ART 这样的运行时系统尤其重要，因为解释器派发、慢路径跳转、链接修补、trampoline 入口和异常回退都大量依赖“先占位、后回填”的控制流组织方式。

在 JIT 后端和运行时入口点中，这类约束会具体表现为三个问题。第一，短分支是否足以覆盖目标块，决定了慢路径布局方式是否需要调整。第二，字面量、入口点地址和 trampoline 地址的构造方式，决定了汇编器是否必须提供统一的标签与修补抽象。第三，若平台相关地址计算路径不稳定，则调试信息、反汇编和崩溃定位都会受到影响。因此，LoongArch 平台的控制转移特点不仅影响“能否发出指令”，更影响执行引擎整体结构是否易于维护和验证。

2.3.3 内存访问、原子操作与同步语义

ART 运行时在对象分配、垃圾回收、读写屏障、锁实现和线程状态切换中高度依赖内存访问与同步语义。LoongArch 在普通访存、原子读改写、屏障和特权控制状态等方面给出了明确的体系结构定义 [3]。对平台适配而言，这意味着运行时中的对象访问、字段更新、锁字操作以及部分 GC 协议必须与架构语义保持一致，否则就可能在并发环境下出现极难复现的功能错误。

相比用户态普通程序，ART 对这些语义的敏感度更高。原因在于它既要处理 Java 层对象模型，又要处理 native 层线程和异常状态；一旦同步原语、内存序或异常边界处理不当，问题往往会表现为“偶发崩溃、偶发错值或偶发性能回退”。因此，LoongArch 的内存与同步语义不是只在锁实现中才重要，而是与执行引擎正确性和优化安全性同时相关的基础约束。

2.4 性能评测与优化问题分析

2.4.1 经典 JVM benchmark 的代表性

性能优化研究离不开具有代表性的 benchmark。对于 JVM 运行时研究，DaCapo 与 SPECjvm2008 是最常见、也最具历史可比性的两类工作负载。前者更接近对象分配、字符串处理和复杂控制流较多的现代 Java 应用，后者则保留了更偏传统数值与吞吐导向的基准场景 [26, 27]。这也是本文后续选择 DaCapo 的 pmd 与 SPECjvm2008 的 mpegaudio 作为核心观察对象的直接原因。

但基准选择本身也具有方法学边界。后续研究表明，不同 benchmark 套件对优化机会的“可见度”并不一致；例如 Renaissance 之所以被提出，正是因为传统套件对现代并发与并行工作负载的覆盖不足，难以暴露某些 JIT 优化或运行时机制的真实价值 [28]。因此，基准结果的解释必须与 workload 特征绑定，而不能把某一子项的收益直接外推为全部应用场景中的普遍结论。

2.4.2 Warmup、波动与统计严谨性

JIT 运行时的性能测量从来不是单次运行取单一数值这么简单。由于解释器驻留时间、编译触发时机、代码缓存状态、GC 行为以及系统噪声的共同影响，Java benchmark 往往具有显著的波动性。统计严谨的性能评测研究指出，若仅使用 best-of-N 或单次结果，就可能把实验噪声误判为优化收益，甚至得出方向错误的结论 [29]。

更进一步，warmup 本身并不总是“快速结束并进入稳定峰值”的简单两阶段过程。相关研究表明，不少 JVM 基准即使在严格受控环境下，也可能出现性能衰退 (slowdown)、无稳态 (no steady state) 等行为；也就是说，丢掉前几轮数据并不总能得到真实、可比的稳态结果 [30]。这对本文尤其重要，因为短生命周期 workload 恰恰更容易让 JIT 阈值、调试信息固定成本和运行时初始化成本放大为主导因素。因此，本文后续在性能分析中始终强调固定 baseline、重复实验和工作负载边界，而不把单轮结果直接上升为一般性结论。

2.4.3 低扰动可观测性需求

优化研究不仅需要跑分，还需要知道“为什么变快或变慢”。这要求实验系统具备足够的可观测性，包括低扰动日志、profile、火焰图和必要的运行时统计信息。Android 运行时研究中的 ARTist、Hubble 等工作表明，若观测机制本身开

销过高，测到的现象很可能已经被工具扰动污染；相反，只有把追踪路径压到足够低的成本，性能调试结果才有解释价值 [31]。

对 LoongArch 平台 ART 而言，这一点同样成立。由于本文研究对象既包含解释器和 JIT 的时机问题，又包含运行时字符串热路径等细粒度优化，单看最终 benchmark 分数远不足以解释方案优劣。必须结合 run.log、符号级热点统计、调用图与火焰图等多种证据，才能建立“热点变化 -> 固定成本变化 -> 最终分数变化”的因果链。这也是第 4 章和第 5 章采用统一数据采集目录和固定分析口径的理论基础。上述方法学原则将在第 4 章中具体落实为本文的实验框架设计。

2.5 AOSP 15 ART 移植的关键技术难点

2.5.1 执行引擎适配范围广且耦合紧密

从工程边界看，ART 并不是一个单独的解释器程序，而是由解释器、JIT 编译器、JNI 编译器、运行时入口点、对象模型、调试支撑和测试基础设施共同组成的复杂系统 [5, 32]。因此，新架构平台的移植工作天然具有“覆盖范围广、模块之间强耦合”的特征。对 LoongArch 平台而言，任何一个模块缺失，都可能导致 Java 到 native、解释执行到编译执行、正常路径到异常路径的调用链无法闭合。

这也是为什么既有论文在总结 JVM/ART 移植经验时，往往都采用“由最小闭环逐步扩展”的实现策略，而不是试图一次性补齐所有高性能路径 [20, 21]。本文后续的实现章节同样遵循这一思路：先让 C++ 解释器成为语义基线，再逐步扩展到 Nterp、JIT、JNI 和快速路径。

2.5.2 正确性验证与性能研究必须并行考虑

对于普通应用程序，编译通过和功能可用往往已经能够说明大部分问题；但对运行时系统而言，编译通过只说明“接口大致接上了”，并不意味着执行引擎已经具备可验证的正确性，更不意味着其性能结论具备研究价值。一方面，解释器、JIT 和运行时入口点中的小错误可能只在极端控制流、异常恢复或对象访问边界上暴露；另一方面，若在存在语义错误的前提下直接进行 benchmark，所有性能结论都会失去解释基础。

因此，新架构 ART 研究必须同时组织正确性验证链路与性能评测链路。前者通过 Host 测试、Target run-test、libcore 测试等方式证明执行语义收敛；后者

则在统一 baseline 和统一工作负载下观察优化收益边界。两者相辅相成，缺一不可。这也是本文把“系统验证方法与实验平台”单列为一章，而不把所有实验内容混写进移植实现章节的原因。

2.5.3 新版本 ART 的复杂性与历史可比性问题

与早期 Dalvik 或早期 ART 相比，AOSP 15 中的执行引擎在解释器实现、JIT 基础设施、调试信息、运行时组件组织以及测试框架上都更加复杂 [5, 6]。这意味着，旧版本平台移植经验虽然仍有参考价值，但不能机械照搬到新版本源码上。尤其是当源码结构、运行时策略和性能瓶颈都已发生变化时，研究工作既要保持与既有论文的历史可比性，又必须基于新版本真实代码路径重新建立问题分析框架。

对本文而言，这种复杂性最终体现为两项要求。第一，移植实现必须以 AOSP 15 当前版本的模块边界和执行机制为准，而不是以历史版本接口为准。第二，性能优化必须建立在新版本正式 baseline 的热点事实上，而不是沿用旧版本经验直接选题。只有这样，论文中的“适配”和“优化”两条主线才能同时具有工程真实性与研究说服力。

2.6 本章小结

本章围绕 ART 的体系结构、解释与编译机制、LoongArch 架构约束、性能评测方法学以及新版本 ART 移植难点，建立了本文后续研究的技术背景与问题分析框架。归纳而言，LoongArch 平台上的 ART 研究既受到执行引擎分层结构的约束，也受到目标架构调用约定、控制转移和内存语义的约束；同时，性能结论又必须建立在统计严谨、低扰动且与 workload 特征一致的实验框架之上。基于这些背景，第 3 章将进一步展开 LoongArch 平台 ART 执行引擎的适配设计与实现。

第3章 LoongArch 平台 ART 执行引擎的适配设计与实现

3.1 总体设计目标与实现策略

相较于普通用户态程序，ART 的平台适配具有更强的系统性。一方面，解释器、JIT 编译器、JNI 桥接与运行时入口点之间存在紧密耦合关系，任何一个子系统缺失都可能导致 Java 方法调用链无法闭合；另一方面，ART 在执行过程中高度依赖体系结构相关细节，例如调用约定、寄存器保存规则、异常现场恢复、分支跳转距离以及线程本地存储访问方式等。因此，本文并未将 LoongArch 平台适配理解为将若干体系结构相关文件逐一翻译到新指令集这一局部任务，而是将其视为执行引擎整体能力的分阶段构建过程 [3, 5, 6, 33, 34]。

本文采用由浅入深、由强可移植性路径向高性能路径逐步推进的实现策略。首先，以 C++ 解释器为起点建立最小可运行闭环。该路径对平台相关组件依赖最少，能够尽早暴露调用约定、栈帧组织、方法调用和异常传播中的基础问题。其次，在解释器路径可稳定运行的前提下实现 Nterp 汇编解释器，以逐条补齐 DEX 字节码处理逻辑的方式提高执行效率，并验证 LoongArch 汇编层面的入口、派发与返回路径。再次，围绕 Optimizing Compiler 补齐 LoongArch64 JIT 后端，使运行时具备从 HIR 到本地机器码的完整转换能力。其后，补齐 JNI 编译器和运行时快速路径，使 LoongArch 平台上的 ART 能够跨越 Java/native 边界并具备完整的运行时入口支撑。最后，扩展 Intrinsic 与标准库热点路径支持，使执行引擎不仅能够运行 Java 代码，而且能够为高频库方法提供足够的机器级实现，形成可用于后续验证与优化的稳定基线。

这种阶段化实现策略与新架构平台上的 JVM 移植经验是一致的 [20, 35]。其核心思想不是一开始就追求所有执行模式同时可用，而是优先保证最小执行路径闭合，再逐步收敛高性能路径上的体系结构相关细节。对本文而言，这种策略带来了两个直接收益：其一，问题定位边界更加清晰，便于将错误限制在单一模块内部；其二，能够尽早建立“可执行、可验证、可回归”的版本，为后续性能优化提供稳定基线。

表 3.1 LoongArch 平台 ART 执行引擎适配的阶段化策略

阶段	目标	主要工作
第一阶段	建立最小可运行路径	补齐汇编器、寄存器描述与 C++ 解释器相关辅助路径，保证简单 Java 程序可以运行
第二阶段	建立高效解释执行路径	实现 Nterp 入口、派发表、栈帧规范与主要 DEX 字节码处理逻辑
第三阶段	建立编译执行路径	实现 JIT 后端、SlowPath、链接修补、OSR 相关支持与调试能力
第四阶段	补齐 Java/native 闭环	实现 JNI 调用桥接、运行时快速路径、异常与对象分配入口点
第五阶段	完善标准库热点支持	扩展 Intrinsic 与运行时能力，形成稳定基线版本

3.2 汇编器与基础代码生成支持

3.2.1 LoongArch64 汇编器实现

汇编器是新架构适配中最底层、但又最关键的基础设施。无论是 Nterp 汇编解释器、JIT 代码生成、JNI 宏汇编器，还是运行时入口点桩代码，最终都需要依赖统一的机器指令发射接口。因此，本文首先完成 LoongArch64 汇编器、寄存器描述与反汇编支持的建设，使其能够支撑上层组件持续演进。

LoongArch64 采用定长 32 位指令格式，寄存器体系和立即数字段划分与 ARM64、x86_64 均有明显差异 [3, 33]。为了在 ART 中形成统一抽象，本文首先补齐了 LoongArch64 的通用寄存器、浮点寄存器和寄存器打印接口，使寄存器分配器、调用约定访问器和调试工具能够共享同一套机器描述。随后，围绕汇编器发射路径逐步实现了整数运算、访存、跳转、屏障、原子操作、浮点运算、移位、时间读取和常用伪指令等指令族。其中，立即数加载、大偏移访存和 PC 相对寻址被优先实现，因为它们同时影响解释器派发、JIT 字面量加载和运行时桩代码生成。

与只面向手写汇编文件的工具链不同，ART 中的汇编器还需要服务于编译器自动生成代码。因此，本文不仅实现了基础指令编码，还提供了适用于编译器

场景的组合式接口，例如任意 32/64 位常量加载、大偏移地址加法、基于标签的分支占位与回填等。这些接口将 LoongArch64 指令编码细节封装在 `Assembler-LOONGARCH64` 内部，使上层代码生成器可以更关注语义而不是位域拼装。

在汇编器基本可用之后，本文进一步补齐了 LoongArch64 反汇编器与配套测试。前者用于在崩溃定位、代码生成验证和 CFG 导出中提供可读汇编；后者则通过逐条比对输出结果验证编码正确性，从而保证后续 Nterp、JIT 和运行时入口点能够建立在可靠的底层发射能力之上。

3.2.2 分支、字面量与链接修补机制

在执行引擎适配过程中，分支与地址修补机制的重要性并不低于普通算术指令。原因在于，ART 的解释器和编译器大量使用标签跳转、异常慢路径跳转、延迟绑定入口点以及字面量池访问，这些都要求汇编器在代码发射时允许“先占位、后回填”的增量式组织方式。

LoongArch64 的控制转移指令在跳转距离上具有明确限制，短分支、长跳转和 PC 相对地址构造的编码方式彼此不同 [3, 34]。为此，本文在汇编器与代码生成器中实现了标签绑定、字面量记录和链接修补三类机制。其核心思想是：在第一次发射分支时仅记录语义信息和占位编码；待目标标签或字面量地址确定后，再在代码整理阶段统一计算真实位移并选择合适的实现形式。这一机制同时服务于如下三个场景。

第一，服务于 Nterp 派发表与辅助函数跳转。解释器中的公共 slow path 和若干 helper 例程需要依赖跨块跳转，如果缺乏统一的标签与分支回填机制，将很难在指令尺寸受限的前提下保持代码可维护性。

第二，服务于 JIT 生成代码中的 SlowPath 和异常路径。诸如空指针检查、边界检查、类初始化检查等路径通常采用“主路径短跳转 + 慢路径延后布局”的组织方式。本文实现的链接修补能力使得 `CodeGeneratorLOONGARCH64` 可以按语义组织控制流，而将最终分支编码选择延迟到 `FinalizeCode` 阶段。

第三，服务于运行时入口点和 trampoline 代码。方法调用、接口冲突表、解析入口点和 deopt 入口往往带有平台特定的长跳转需求。通过统一的标签、字面量和链接修补抽象，本文避免了在不同模块中重复实现同类地址计算逻辑，提高了执行引擎各子系统之间的一致性。

3.3 解释器适配

3.3.1 C++ 解释器适配

C++ 解释器是本文建立 LoongArch 平台最小可运行路径的起点。与 Nterp 和 JIT 相比, C++ 解释器对平台特定汇编能力的依赖最少, 其核心执行循环主要由平台无关的 switch-case 逻辑构成, 因此能够尽早暴露方法调用、参数传递、ShadowFrame 布局以及异常传播中的底层问题。

尽管解释器主体使用 C++ 编写, 但从 `ArtMethod::Invoke()` 进入解释执行仍涉及多项平台相关内容。首先, 需要保证 LoongArch64 调用约定与 ART 的栈帧组织方式一致, 使解释器入口可以正确接收 `Thread*`、`ArtMethod*`、参数区和返回值位置等关键信息。其次, 需要补齐与解释器配套的汇编辅助路径, 例如静态/直接调用桩代码、方法返回寄存器恢复和部分运行时入口。再次, 需要确保 DEX PC、调用现场和异常状态能够在 LoongArch64 上被正确保存与恢复, 否则即使简单方法能够执行, 异常处理和调试也无法稳定工作。

在实现顺序上, 本文首先保证最简单的 Java 程序能够走通“方法调用、解释执行、结果返回”的闭环, 随后逐步补齐对象分配、字段访问、类型检查和异常处理相关路径。这样做的目的不是长期停留在低性能解释执行, 而是借助最可移植的执行模式为后续 Nterp 和 JIT 提供语义基线。换言之, C++ 解释器在本文中既是功能起点, 也是后续高性能路径的重要参照系。

3.3.2 Nterp 框架建立

在 C++ 解释器具备基本可运行能力后, 本文继续实现 LoongArch64 下的 Nterp 汇编解释器。Nterp 是 ART 中面向热点解释执行的高性能路径, 其基本思想是通过预先组织好的派发表, 将 opcode 直接映射到对应的汇编处理逻辑, 从而减少传统 switch-case 解释器中的分支分派开销。由于该路径高度依赖平台汇编实现, 因此也是新架构适配中最能体现体系结构细节的部分。

LoongArch64 Nterp 的建立首先需要定义专用 ABI 约束, 包括线程寄存器、解释器帧指针、当前 DEX PC、当前指令缓存和派发表基址等关键寄存器的固定用法。与普通函数调用不同, Nterp 的目标是在解释器主循环内部尽量避免多余保存/恢复开销, 因此寄存器分工必须在入口处一次性建立, 并在后续 opcode 处理逻辑中保持稳定。为此, 本文首先实现 Nterp 框架骨架, 包括入口函数、栈帧

建立、参数装载、类初始化路径以及与 C++ 解释器之间的共享帧布局。这种围绕固定寄存器协议和直接分派组织解释器热路径的做法，与高效解释器研究中对分派结构和寄存器约束的结论是一致的 [16, 33]。

在入口建立之后，本文进一步补齐 Nterp 的派发表组织方式。具体而言，LoongArch64 平台通过“固定宽度处理块 + 派发表基址”构造字节码到汇编代码的快速映射关系，使常见指令能够以较低分派成本进入对应处理例程。对于超出单个处理块容量或需要复杂逻辑的 opcode，则通过 helper 和 slow path 方式进行扩展。这种组织方式一方面保证了解释器主路径紧凑，另一方面也使复杂语义可以在不破坏整体结构的前提下逐步补齐。

3.3.3 Nterp 指令补齐与问题修复

Nterp 的难点并不在于建立入口骨架，而在于将 200 余条 DEX 字节码处理逻辑逐步补齐，并在这一过程中保证与 C++ 解释器完全一致的语义。本文在实现时采用“按指令类别逐步闭环”的方式推进：首先补齐常量、返回、移动和整数算术等基础指令，使最小测试用例可以运行；随后继续实现浮点运算、静态字段访问、实例字段访问、数组访存、对象创建、类型检查和类初始化相关指令；最后补齐各类 invoke 及其 range 变体，并引入 nterp-to-nterp 快路径和必要的 slow path。

这一迭代过程中，本文集中解决了若干具有代表性的 LoongArch64 平台问题。其一，Nterp 与调用桩代码之间的栈帧衔接问题。如果解释器栈与 native 栈对齐规则不一致，方法调用返回时就可能发生栈破坏，进而导致 invoke 返回路径崩溃。本文通过统一对齐策略和保存/恢复顺序，修复了 invoke stub 返回后的栈破坏问题。其二，浮点 NaN 分类问题。LoongArch64 在浮点比较和分类上的表现与既有平台存在差异，若直接沿用其他架构的判断路径，可能导致部分浮点测试出现不一致。本文针对该问题补充了平台化修正。其三，JIT 与 Nterp 混合执行时的对象状态问题。随着 JIT 路径逐步可用，Nterp 不再是孤立执行路径，其与编译代码之间的对象与寄存器状态切换必须保持一致。本文进一步修复了 JIT invoke 场景下的对象损坏问题，确保解释执行与编译执行能够在同一运行时环境中稳定共存。

通过上述逐类补齐与问题修复，LoongArch64 Nterp 最终从“仅能运行极简样例”的骨架逐步演化为能够覆盖完整 DEX 字节码集的汇编解释器。这一结果

意味着 LoongArch 平台不仅具备了平台无关的解释执行能力，也具备了与主流架构接近的高性能解释执行通路。

3.4 JIT 编译器后端适配

3.4.1 LoongArch64 代码生成器实现

如果说解释器适配解决的是“代码尚未编译时如何执行”的问题，那么 JIT 后端适配解决的就是“热点代码如何生成本地机器码”的问题。ART 的 Optimizing Compiler 以 HIR 为核心中间表示，前端负责构建 HGraph 并执行平台无关优化，平台后端则负责寄存器分配、调用约定映射和机器指令发射。因此，LoongArch64 JIT 后端的核心任务，是将 ART 已有的优化编译基础设施同 LoongArch64 的寄存器、指令和调用约定约束连接起来 [32–34, 36]。

本文围绕这一目标，首先补齐了 LoongArch64 的代码生成器类、寄存器阻塞规则、调用约定访问器和栈帧管理逻辑。具体而言，CodeGeneratorLOONGARCH64 负责统一组织代码发射，InstructionCodeGeneratorLOONGARCH64 负责按 HIR 指令生成平台相关机器代码，调用约定访问器则负责在 Java 调用、运行时调用和返回值传递之间建立一致的寄存器/栈映射关系。在这一基础上，本文进一步实现了并行移动、参数装载、返回寄存器恢复、帧建立与销毁、显式/隐式空指针检查以及若干辅助工具接口，使 LoongArch64 后端可以接入 ART 的寄存器分配与代码布局流程。

3.4.2 核心 HIR 节点支持

JIT 后端适配的主体工作，是将平台无关 HIR 节点映射为可执行的 LoongArch64 指令序列。本文按照从基础到复杂的顺序逐步补齐相关节点支持。首先，完成整型/长整型/浮点常量、算术运算、移位、比较、分支、返回和参数节点等基础 HIR，使简单方法具备基本编译执行能力。随后，继续补齐数组长度、边界检查、字段读写、类加载、异常值装载和方法句柄等更接近真实应用的节点，以支持常见对象与数组操作。最后，围绕静态调用、直接调用、虚调用、接口调用、未解析调用以及自定义/多态调用等复杂语义补齐 invoke 族节点，使 LoongArch64 后端能够覆盖 ART 中最关键的控制流与调用场景。

在这一过程中，本文特别重视“主路径代码生成”和“慢路径处理”之间的协同组织。对于空指针、越界、类初始化和异常抛出等语义，JIT 后端并不直接在

主路径中展开复杂处理，而是采用主路径短检查加慢路径回退的方式降低正常执行开销。为实现这一点，本文在 LoongArch64 后端中配套实现了必要的 linker patch、literal 访问、运行时调用封装和 SlowPath 对象布局机制，使热点路径与异常路径能够在统一框架下共存。

3.4.3 OSR 与运行时协同支持

JIT 后端闭环不仅要求“能生成代码”，还要求所生成代码能够与运行时共同维护热度、去优化和异常恢复等动态行为。为此，本文进一步补齐了 On-Stack Replacement (OSR)、去优化、方法进入/退出钩子以及内联缓存更新相关支持 [18, 19]。其中，OSR 允许热点循环在执行过程中从解释器切换到已编译代码，是 ART 分层执行机制中的关键环节；去优化则保证在假设失效或运行时状态变化时，已编译代码可以安全回退至解释执行。

在实现层面，本文为 LoongArch64 平台接入了必要的运行时入口和协同逻辑，包括热度递增、内联缓存更新、去优化入口、字符串构建辅助、快速方法调用和崩溃定位所需的反汇编能力等。这样一来，LoongArch64 JIT 后端不再只是静态的代码发射工具，而是能够真正参与 ART 的动态执行生态，在解释执行、编译执行与运行时状态切换之间保持一致行为。

3.5 JNI 编译器与运行时快速路径实现

3.5.1 JNI 调用桥接实现

在 ART 中，Java 代码与 native 代码之间的调用并不直接遵循普通 C 函数调用语义，而是需要经过专门的 JNI 桥接层完成线程状态切换、参数整理、栈帧组织与异常传播。对于新架构平台而言，若这一桥接层没有完整实现，即使解释器与 JIT 都已可用，Java 方法也仍无法稳定调用 native 代码。因此，JNI 编译器和 JNI 宏汇编器是执行引擎闭环中不可缺少的一环。

本文首先实现了 LoongArch64 JNI 调用约定描述，使 Java 参数、返回值、JNIEnv* 与隐藏参数可以按平台规则在寄存器和栈之间分配。随后，以此为基础补齐了 JNI macro assembler 骨架，并逐步实现保存现场、构建 JNI 帧、线程状态从 Runnable 切换到 Native 以及从 Native 返回 Runnable 的核心路径。在宏汇编器稳定后，本文进一步接入普通 native 方法的 JNI 编译支持，并补齐栈展开相关的 WriteCIE() 需求，使 LoongArch64 平台能够在异常与调试场景下正确恢复调用链

[33]。

需要指出的是，JNI 路径的适配原则并非一开始就追求最激进的 native 快路径。本文在适配阶段优先保证普通 JNI 调用与基本 native 方法执行闭环，对更激进的 direct @CriticalNative 路径则采取保守门控策略。这样做的原因在于，该路径对调用约定、异常处理和返回值封装一致性要求更高；在平台化适配尚处于收敛阶段时，优先保证功能正确性比贸然放开高风险快路径更为稳妥。

3.5.2 运行时入口点与快速路径补齐

JIT 和解释器都高度依赖运行时入口点提供服务。对象分配、锁操作、字段访问、类解析、类型检查、异常抛出和方法调用等高频操作，最终都需要通过 art_quick_* 入口点或相关 trampoline 落到运行时实现上。因此，运行时快速路径的补齐并非附属工作，而是 LoongArch64 执行引擎真正可用的必要条件。

围绕这一目标，本文逐步实现和接入了对象分配入口点、CheckInstanceOf 相关入口、快速字段访问入口、快速方法调用桩、类解析入口、字符串相关入口、deopt 入口以及数学辅助入口等关键组件。与此同时，本文还补齐了与方法分派和异常恢复相关的支撑模块，例如 ImtConflictTable trampoline、SignalHandler 和部分故障恢复路径，使编译代码和解释器在面对接口冲突、信号异常和运行时错误时均能够回到受控处理流程。

从工程角度看，这部分工作的价值在于将分散的体系结构相关函数组织成统一的运行时支撑层。只有当解释器、JIT 和 JNI 都能够共享这一套运行时入口点时，LoongArch 平台上的 ART 才具备与主流架构相同的执行语义基础。

3.6 Intrinsic 与执行引擎能力扩展

在完成解释器、JIT 和 JNI 主体闭环后，LoongArch 平台仍面临一个容易被忽视但实际非常关键的问题：即 Java 标准库中的热点方法在新平台上是否具备足够的执行支持。ART 中大量热点方法依赖 Intrinsic 实现，以便在编译阶段直接映射为更短、更高效的机器代码。若新架构缺失这些支持，虽然程序仍可通过常规调用路径运行，但在性能和兼容性上都会与成熟平台存在明显差距。

为此，本文进一步实现并扩展了 LoongArch64 相关 Intrinsic 与运行时能力，包括 Memory peek/poke、整型/浮点最值计算、Float 与 Double 相关 Intrinsic，以

及 `Math.fma` 等典型数值计算路径。与此同时，本文还根据 AOSP 15 的演化情况同步扩展了运行时支撑，修复和补齐了若干原本仅在其他架构上完整可用的热点路径。

需要强调的是，本文在适配阶段对 `Intrinsic` 的处理原则是“优先补齐能力，再讨论进一步优化”。也就是说，这一阶段的目标首先是确保标准库热点路径在 LoongArch 平台上具备正确、稳定的机器级实现，而不是立即将所有平台特化收益都压榨到极致。这一保守而渐进的策略，为后续在稳定基线之上继续开展执行引擎优化提供了更可靠的起点。

3.7 阶段性结果分析

从阶段结果来看，本文的适配工作呈现出明显的逐层闭环特征。最初，LoongArch 平台仅具备零散的机器描述和部分汇编能力，尚不足以支撑 Java 方法执行。随着汇编器、C++ 解释器和基础运行时路径补齐，系统首先具备了最小可运行能力。随后，`Nterp` 从骨架逐步扩展到完整字节码覆盖，解释执行路径从能够运行基本样例提升到“可承担主力解释执行”。在此基础上，JIT 后端、JNI 编译器和运行时快速路径相继闭合，使编译执行与 Java/native 调用链也具备了可用性。最终，通过 `Intrinsic` 和若干保守门控策略的补充，LoongArch 平台上的 ART 执行引擎形成了可用于系统验证与后续性能研究的稳定基线。

从研究对象边界看，本文将从初始移植起点到稳定基线版本这一提交区间视为“执行引擎适配阶段”的主体成果。这一阶段工作的核心价值，不在于单个局部优化点，而在于系统性补齐了 LoongArch 平台上 ART 执行引擎所需的关键构件，使后续的正确性验证和性能优化建立在完整、可复现的工程基础之上。

3.8 本章小结

本章围绕 LoongArch 平台 ART 执行引擎适配，系统介绍了汇编器与基础代码生成能力、C++ 解释器与 `Nterp` 汇编解释器、JIT 编译器后端、JNI 调用桥接、运行时快速路径以及 `Intrinsic` 扩展等方面的实现工作。这些工作共同构成了新架构平台上 ART 的完整执行基础。其结果表明，只有将解释执行、编译执行、Java/native 桥接与运行时支撑作为一个整体来适配，LoongArch 平台上的 ART 才能从“局部可运行”演进为“系统可验证、可优化”的完整运行时。下一章将在

此基础上进一步给出正确性验证方法、实验平台组织方式以及正式 baseline 的建立过程。

第 4 章 系统验证方法与实验平台

4.1 实验环境与软硬件平台

为验证 LoongArch 平台 ART 执行引擎适配结果的正确性与可用性，本文在真实目标设备上建立了完整的编译、部署、回归与基准测试实验链路。整体实验平台基于龙芯 3A6000 处理器，采用四核、主频 2.5GHz、16GB 内存的硬件配置，目标系统为 Android 15，内核版本为 Linux 6.6.46。软件侧以 AOSP 15 源码中的 art/ 子项目为研究对象，围绕 LoongArch64 平台补齐执行引擎相关模块，并在此基础上开展后续的正确性验证与性能分析。

本文在验证阶段统一采用 LoongArch64 的 64 位调试构建口径。一方面，调试构建更便于暴露执行引擎适配过程中潜在的调用约定错误、慢路径跳转错误和异常恢复问题；另一方面，本文后续三项正式性能方案也建立在相同的调试构建环境之上，因此在验证章节保持同一执行口径，有助于保证论文前后实验环境的一致性。

需要指出的是，本文关注的是 LoongArch 平台 ART 执行引擎的系统适配与执行行为，而非单个组件的孤立验证。因此实验平台的组织原则是：所有验证步骤均围绕“同一套编译产物、同一台目标设备、同一条执行链路”展开，避免由于版本漂移或部署对象不一致而导致结果失真。

4.2 编译、部署与测试流程

4.2.1 编译流程

ART 的平台适配工作涉及解释器、JIT 编译器、JNI 编译器与运行时入口点等多个子模块，因此本文在功能验证和性能实验前均优先执行完整 ART 编译，以确保所有相关动态库、桩代码和目标机测试产物处于同一源码版本下。为避免将作者个人工作站路径误写为统一实验条件，本文用 AOSP_ROOT 表示实验所使用的 AOSP 15 源码树根目录；在任意复核环境中，只需将其替换为本地源码路径即可沿用同一编译流程。本文采用的统一编译入口可写为：

```
cd ${AOSP_ROOT} && art/tools/buildbot-build.sh --target
```

采用统一编译入口的原因有两点。首先，ART 的许多平台相关问题不会只出现在单个动态库中，而是会同时影响 `libart.so`、`libartd-compiler.so`、测试用运行时环境以及若干目标机辅助组件；若仅进行局部增量编译，容易出现宿主侧与目标机侧产物不一致的问题。其次，完整编译能够在较早阶段暴露链接错误、未实现符号和部分配置不匹配问题，从而降低后续部署与测试阶段的干扰项。

4.2.2 部署流程

完成编译后，本文默认采用轻量部署方式将新的 ART 产物推送到目标设备。需要说明的是，部署步骤与设备分区挂载方式、构建输出目录以及调试权限配置紧密相关，因此正文不再把作者本地维护的宿主机侧同步脚本写成“统一命令”，而只保留其对实验结论真正重要的流程约束：从本次构建输出中收集 ART 相关库、可执行文件和测试产物，再通过 `adb push`、`adb sync` 或等价同步机制覆盖到目标设备对应路径，最后在相同设备状态下执行回归与基准测试。

轻量部署的核心目标是只同步与 ART 执行链路直接相关的关键产物，减少完整系统镜像推送带来的额外时间成本与环境扰动。只有在明确涉及系统镜像、框架依赖或其他重量级组件同步时，才使用全量推送方式。对本文而言，大多数执行引擎适配和优化实验均可通过轻量部署完成，这也使得“修改、编译、推送、验证”的迭代周期保持在可接受范围内。

需要强调的是，目标机回归和基准测试均建立在部署后的真实设备环境之上。若未先推送新产物，随后执行 `art/test.py` 或基准测试脚本得到的结果将无法反映当前源码状态。因此，本文在实验链路上始终遵循“编译、推送、目标机回归、基准测试”这一固定顺序。

4.2.3 验证链路组织方式

本文将所有验证过程组织为四级链路：

1. **编译校验**：确认平台相关代码在当前源码状态下能够完整构建。
2. **部署同步**：将新编译产物推送至目标设备，保证待测环境与源码版本一致。
3. **目标机回归**：执行最小相关 `run-test` 或功能性测试，验证语义正确性与稳定性。
4. **基准测试评测**：在已确认正确的版本上采集性能数据、日志和火焰图。

这种组织方式的价值在于将“功能正确”和“性能可比”严格分离。只有在前一层验证通过后，下一层实验结论才具有解释价值。特别是在执行引擎适配与性能优化并行推进的过程中，这一链路可以避免把由语义错误引发的崩溃、回退或异常抖动误判为性能问题。

4.3 正确性验证方法

4.3.1 宿主机侧测试

宿主机侧测试的作用，是在不依赖目标设备的前提下尽早验证 ART 基础功能是否受到平台化改动影响。本文主要使用 ART 自带的 Host GTest 与相关单元测试集，覆盖 DEX 文件处理、命令行解析、编译器基础设施、运行时公共库等子系统。此类测试虽然不能直接证明 LoongArch 目标机上所有执行路径都正确，但能够有效筛查平台适配过程中引入的通用性回归，例如 DEX 验证器行为变化、编译器公共组件失效或命令行参数处理异常。

从作用定位上看，宿主机侧测试更接近前置过滤环节。它不能替代目标机验证，但可以在进入设备侧实验前提前排除一批明显错误，从而减少目标机调试的复杂度。

4.3.2 目标机侧 run-test 验证

目标机侧 run-test 是本文验证执行引擎语义正确性的核心手段。ART 测试框架会将测试用例编译为 DEX，并把动态库、测试程序与运行脚本组织到设备侧测试沙箱中执行，从而覆盖解释执行、编译执行、异常处理、对象访问和调用约定等关键行为。

在日常定位中，本文优先采用最小相关测试集合进行回归。例如，对执行引擎和 JIT 相关改动，常用命令为：

```
1 ./art/test.py -r --target --no-prebuild --debug --no-image --64  
   --jit -j 1 -t <case>
```

当需要验证平台适配阶段的整体正确性时，则使用更大范围的目标机测试集合，分别在 C++ 解释器、Nterp 解释器和 JIT 编译器三种模式下执行。由于三种模式下的全量命令矩阵较长，正文仅保留按执行模式整理后的汇总结果；对

应命令、结果摘要与原始日志统一保存在外部实验数据目录与统计记录中。按当前归档统计，三种执行模式下均完成了 1033 个测试用例的验证，通过情况如表 4.1 所示。

表 4.1 目标机侧 run-test 验证结果

执行模式	通过测试数	通过率
C++ 解释器	1033/1033	100%
Nterp 解释器	1033/1033	100%
JIT 编译器	1033/1033	100%

这一结果说明，LoongArch 平台 ART 执行引擎已经不仅能够运行简单样例，而且在三条主要执行路径上均通过了系统性的目标机回归，具备进入基准测试评测阶段的语义基础。

4.3.3 libcore 测试

仅依赖 run-test 仍不足以证明 Java 核心类库层面的行为完全正确，因此本文进一步引入 libcore 测试集，用于覆盖集合框架、输入输出、并发、数学运算、加解密与网络等 Java API 语义。本文采用设备侧运行方式执行 libcore 测试。按当前实验记录保留的包级结果摘要，设备侧已覆盖 libcore 的主要测试包，验证结果以通过为主，少量测试因架构条件限制被自动跳过。现有归档保留的是包级结果摘要而非逐包统计明细，因此本文不再将其写成精确通过率数字；这些跳过项不涉及执行引擎核心语义，不影响主要正确性结论。

与 run-test 相比，libcore 测试的意义在于：前者更贴近运行时和执行引擎行为，后者则更贴近 Java 应用层面对 API 语义的观察结果。两者结合后，才能较完整地证明 LoongArch 平台 ART 已具备作为 Android 应用运行时的基本正确性。

4.3.4 差分测试与辅助调试方法

在执行引擎适配过程中，单纯依赖“通过或失败”的测试结果往往不足以快速定位错误来源，因此本文还建立了若干辅助调试方法。

首先，针对解释器路径，本文采用差分测试方法，将同一段 Java 程序分别在参考平台和 LoongArch 平台的解释执行环境中运行，通过追踪每条字节码执行后的寄存器与对象状态，定位首次出现偏差的 DEX PC。该方法尤其适用于对

象访问、字段读写和类型检查等语义细节问题的定位。

其次，针对 Nterp，本文采用逐类指令递增验证策略。即在尚未补齐全部 opcode 前，先在运行时中过滤未实现指令，仅对已实现指令类别开放 Nterp 路径；每新增一组指令，即与 C++ 解释器结果进行对照，缩小错误定位空间。这种方法有效降低了汇编解释器一次性大规模实现带来的调试难度。

再次，针对 JIT 后端，本文结合 CFG 导出、反汇编输出和运行时日志进行定位。通过查看控制流图中 HIR 节点、块间跳转和最终生成的 LoongArch64 汇编，可以较快判断问题是发生在前端图构建、后端指令选择，还是运行时入口点调用阶段。

4.4 基准测试设计与正式基线建立

4.4.1 基准测试选择原则

性能实验部分的目标，不是给出泛化至所有 Android 应用场景的绝对结论，而是在统一实验口径下观察执行引擎相关优化对典型工作负载的影响。基于这一目标，本文选择 DaCapo 与 SPECjvm2008 作为主要基准测试集合 [26, 27]。前者更偏向现代 Java 应用中的对象分配、字符串处理与复杂控制流行为；后者则包含更靠近经典 JVM 计算与数值处理路径的负载。

与此同时，本文在基准测试结果解释上明确参考统计严谨的 Java 性能评测方法与 warmup 行为研究，避免将单轮波动或暂时性升降误判为稳定收益 [29, 30]。对于基准覆盖范围本身，本文也有意识地将结论约束在当前工作负载边界内，而不将其直接外推为所有现代 JVM 应用的普遍规律；这与 Renaissance 等后续基准测试工作对“传统套件覆盖有限”的提醒是一致的 [28]。

在具体子项上，本文在正式图表中保留五个工作负载，分别为：

- DaCapo/pmd
- SPECjvm2008/mpegaudio
- Scimark 子项：lu.small、sor.small 和 fft.small

其中，pmd 用于观察解释器驻留、JIT 触发时机与字符串运行时热点；mpegaudio 用于提供与 pmd 明显不同的短生命周期吞吐工作负载；lu.small 是本文用于验证 LoongArch64 LSX 浮点 SIMD 方案的主要受益对象；sor.small 与 fft.small 则用于观察该类数值优化在相邻工作负载上的可迁移性与副作用边界。通过这样的五

类工作负载组合，本文能够避免把某一类局部收益误写为适用于所有基准测试的普遍结论。

4.4.2 正式基线口径

为避免性能结论受到设备状态、命令差异或临时试跑数据的干扰，本文对基准测试实验采用固定口径，并明确区分正式单次比较数据与重复测量验证数据。

- 2026-03-13 final-comparison：用于正文主表和主图的正式单次对比口径。同日基线为：pmd=3272 msec、mpegaudio=25.38 ops/m、lu.small=46.22 ops/m、sor.small=24.76 ops/m 和 fft.small=76.43 ops/m。三项单方案结果均相对这组基线计算。

- 2026-03-14 statistical-validation：用于重复测量验证的独立数据集，仅检验方案一在 pmd 上的收益方向和方案三在 lu.small 上的稳定收益，其结果单独展示，不与前一日的正式单次结果混算。

其中，DaCapo 的评价指标为执行耗时，单位为 msec，数值越小表示性能越好；SPECjvm2008 的评价指标为 ops/m，数值越大表示吞吐越高。本文在后续分析中始终遵循这一指标方向，不将不同基准测试的分值含义混为一谈。

需要说明的是，上述 final-comparison 与 statistical-validation 目录中的 run.log、perf.data、report_symbol.txt、report_callgraph.txt 和 flamegraph.svg 等原始产物，连同对应结果摘要与统计报告，统一保存在外部实验数据目录中，用于论文写作、交叉核验与历史记录；考虑到仓库提交范围与数据体积限制，当前论文仓库未附带完整 test-data/ 目录。因而，论文正文中出现的正式性能结果若无特殊说明，均指上述两类固定实验口径及其导出的分数与热点统计，而不是某个随仓库公开发布的目录快照。

4.4.3 性能数据采集方式

为保证性能分析不仅停留在单一分数层面，本文在基准测试运行时同步采集原始日志与热点数据。每次正式实验目录中通常保留如下文件：

- run.log：记录宿主机侧调用命令、解析后的 dalvikvm64 命令与远端 shell payload；
- perf.data：采集得到的原始采样数据；
- report_symbol.txt：符号级热点汇总；

- `report_callgraph.txt`: 调用图视角的热点信息;
- `flamegraph.svg`: 火焰图, 可直观反映热点路径分布。

通过同时保留分数、日志与 `profile` 数据, 本文在后续优化分析中可以建立更完整的因果链, 即从“分数变化”进一步追溯到“热点分布变化”和“固定成本变化”, 从而避免仅凭单次跑分对方案优劣做出过度推断。这种“尽量降低观测扰动、同时保留足够因果证据”的思路, 也与 Android 场景下低扰动性能调试研究的方法论相吻合 [31]。

4.5 有效性威胁与实验控制

尽管本文已经通过固定编译、固定部署顺序、目标机最小相关回归以及统一基线归档来控制实验变量, 但面向 JIT 运行时的性能研究仍然不可避免地受到若干有效性威胁影响。为避免在第 5 章中对实验结论做出超出证据边界的推断, 本文在写作中对以下因素予以明确约束。

1. **设备状态与后台负载扰动**。基准测试结果会受到目标机温度、后台服务活动、CPU 频率波动和设备前序运行历史的影响。因此, 本文不混用跨日期、跨设备状态的基线, 而是将 2026-03-13 的正式单次对比数据与 2026-03-14 的重复测量数据严格分离, 避免把环境漂移误写成方案收益。

2. **debug 构建口径限制**。本文的正式实验统一建立在 LoongArch64 debug ART 环境下。这一口径有利于在移植收敛阶段保留更多调试信息与验证能力, 但也会放大 JIT 调试信息、符号处理和部分运行时固定成本。因此, 正文中所有性能结论均被限制为“固定 debug 基线下的阶段性结果”, 不直接外推到 release 构建或其它系统配置。

3. **重复测量样本数有限**。本文在第 5 章仅对最关键的两条性能主张进行了有限轮次的同日重复测量, 其目的在于区分“可稳定复现的收益”与“仍需继续验证的收益方向”, 而不是进行严格统计显著性检验。因此, 论文中采用原始散点、均值和标准差来表达结论强度, 而不滥用显著性术语。

4. **JIT 符号可观测性有限**。部分实验归档中存在 `perf-*.map` 缺失或 JIT Java 符号不完整的情况, 这意味着火焰图和 `report_symbol.txt` 更适合用于判断“热点路径是否被命中”和“固定成本是否相对变化”, 而不适合支撑过于细粒度的 Java 方法级归因。因而, 本文在第 5 章中始终将分数变化、回归结果和 `profile` 证据结

合解释，而不单独依赖某一类可观测结果下结论。

4.6 验证结果汇总

4.6.1 移植阶段验证结果

结合第 3 章的实现工作和本章给出的验证链路，可以将 LoongArch 平台 ART 执行引擎适配阶段的结果概括为三点。

第一，宿主机侧测试验证了平台适配未破坏 DEX 处理、命令行解析和运行时公共组件等基础功能，为设备侧回归提供了前置保证。

第二，目标机侧 run-test 在 C++ 解释器、Nterp 和 JIT 三种执行模式下均通过了 1033/1033 的测试集，说明 LoongArch 平台执行引擎的三条主要执行路径在语义层面已经收敛。

第三，libcore 测试集的通过表明，LoongArch 平台 ART 不仅在运行时内部测试上具备正确性，也能够支撑 Java 核心类库层面的实际 API 语义。

这三类结果共同表明：第 3 章所建立的执行引擎能力已经从“可运行”推进到“可系统验证”，为后续性能研究提供了可靠基础。

4.6.2 优化阶段验证要求

性能优化阶段并不重复执行全量系统验证，而是遵循“最小相关回归优先”的原则。即每个候选优化方案在进入基准测试实验前，先根据改动所在模块选择直接受影响的目标机测试集合。例如，字符串热路径优化会优先验证字符串相关 checker 测试和 JIT 回归；JIT 参数调整方案则优先验证 JIT 缓存、JNI stub 和编译相关用例。

这一做法的目的，是在维持验证效率的同时保证结论可信度。若最小相关测试未通过，则基准测试结果不具有解释价值；只有在相关回归通过后，后续性能差异才可以归因于方案本身，而不是功能错误或未定义行为。

4.7 本章小结

本章从实验环境、编译部署链路、正确性验证方法、基准测试组织方式以及正式基线口径等方面，建立了本文后续性能研究所依赖的统一实验框架。与第 3 章偏重模块实现不同，本章强调的是如何证明这些实现的可信性，并为下一章的

优化实验提供统一的数据来源、评价标准和解释边界。

第 5 章 面向 LoongArch 平台 ART 的性能优化研究

5.1 优化目标与分析方法

在第 3 章完成执行引擎适配、第 4 章建立统一验证链路之后，LoongArch 平台上的 ART 已具备稳定、可复核的实验基础。基于这一前提，本文进一步面向 LoongArch 平台 ART 开展性能优化研究。与“先追求绝对最优、再解释原因”的做法不同，本文将优化过程组织为“固定 baseline、热点分析、候选筛选、最小相关回归、benchmark 复测、失败归档”的闭环，以保证每一项结论都可以追溯到统一的实验口径与可解释的热点变化 [6, 17]。

结合第 4 章的实验平台设定，本文的优化研究统一建立在 LoongArch64 64 位 debug 构建和固定 JIT baseline 之上。之所以不在研究阶段频繁更换 baseline，主要有两方面考虑。其一，ART 属于典型的分层执行系统，解释器、baseline JIT、优化重编译以及运行时辅助路径之间存在相互影响；若 baseline 持续漂移，则很难比较不同方案之间的真实边际收益。其二，本文的目标并非给出“对所有 Android 应用都成立”的普适结论，而是在统一口径下识别 LoongArch 平台 ART 当前最值得保留的优化方向，因此更需要强调对照关系和可复核性 [14, 15, 24, 26, 27, 37]。

需要进一步说明的是，本文在第 5 章中同时使用两类实验归档。第一类是方案筛选阶段保留下来的 baseline flamegraph 与热点摘要，它们主要用于解释为何将某条路径提升为候选优化方向；第二类是 2026-03-13 同日采集的 final-comparison 正式单次对比数据，以及 2026-03-14 独立采集的 statistical-validation 重复测量数据，它们共同构成本章正式图表与最终结论的主口径。前者用于热点定位，后者用于收益判定与稳定性讨论，两者在写作中明确区分，不做交叉混算。

为兼顾正文可读性与实验可复核性，本文在正文中统一使用语义化版本名，不直接展开源码提交哈希；相关版本对应的具体源码状态、脚本与数据目录统一保存在实验归档与 benchmark 报告中。本文涉及的主要版本名如表 5.1 所示。

在候选方案筛选上，本文遵循以下三项原则。

1. **优先命中正式 baseline 的主导热点。**仅当火焰图、符号汇总或日志能够证明某条路径持续出现在正式 workload 中，才将其提升为候选优化方向。

表 5.1 本文主要实验版本的语义化命名

版本名	用途	主要出现位置
正式基线版本	第 5 章正式单次对比的统一对照版本	三项正式方案与叠加分析
方案一正式实现版本	JIT 分层热度阈值调整版本	方案一单独实验
方案二正式实现版本	运行时字符串搬运批量化版本	方案二单独实验
方案三最小 SIMD 正式版本	正式方案三采用的最小可用 LSX 浮点 SIMD 版本	方案三正式实验与重复测量
关闭 Intrinsic 的候选基线	候选消融中的无 Intrinsic 对照版本	Intrinsic 消融与完整 SIMD 候选消融
候选链最小 SIMD 版本	候选消融中的最小 SIMD 状态 (S1)	完整 SIMD 候选消融
完整 SIMD 覆盖候选版	补齐更广 SIMD 能力的候选状态 (S2)	完整 SIMD 候选消融
附录阶段链独立基线版本	DexFileVerifier 启动期实验的独立对照版本	附录阶段链实验

2. **优先保留低副作用、可门控的改动。**对执行引擎这类系统软件而言，过度放大全局编译行为或随意改写运行时策略，往往会放大验证成本与性能波动。因此本文更倾向于选择作用域明确、开关边界清晰的方案。

3. **先通过最小相关回归，再讨论 benchmark 收益。**若相关 run-test 未通过，则 benchmark 分数不具备解释价值。只有在语义正确的前提下，性能差异才可归因于方案本身。

基于这一方法，本章最终保留三项正式方案：JIT 分层热度阈值调整方案、运行时字符串搬运批量化方案，以及 LoongArch64 LSX 浮点 SIMD 向量化方案。其中，前两项主要面向短生命周期 workload 的 JIT 进入时机和字符串热路径，第三项则面向规则浮点数值内核的后端向量化能力补齐。同时，为避免论文结论只呈现“成功案例”而缺乏方法论边界，本文还选取若干与上述正式方案在技术方向上密切相关、但最终未形成收益的失败方案进行对照分析。这种写法一方面能够展示优化过程中的筛选逻辑，另一方面也能更清楚地说明 LoongArch 平台 ART 在当前版本中的真正瓶颈所在。

5.2 基线工作负载特征分析

5.2.1 DaCapo pmd 特征

本文选择 DaCapo/pmd 作为核心观测对象之一，原因在于该 workload 在 LoongArch 平台 ART 上同时暴露了解释器驻留、JIT 固定成本和运行时字符串热点三类问题。在方案筛选期用于热点定位的 baseline flamegraph 归档中，pmd 已稳定表现出这些成本特征。对其归档中导出的 report_symbol.txt 进行汇总，可得到表 5.2 所示的主要热点。

从表 5.2 可以看到，pmd 的时间并没有集中在某个单一数学核心循环上，而是同时受到三类成本影响。第一类是解释器热点，例如 INSTANCE_OF、RETURN_OBJECT 和 ExecuteSwitchImplCpp。这说明在当前 debug JIT 口径下，pmd 的一部分热点方法仍有较长时间停留在解释执行阶段。第二类是 JIT 调试信息相关固定成本，例如 ElfDebugReader 与 VisitFunctionSymbols。这类热点在短生命周期 workload 中尤其显著，因为一次性编译与符号处理开销不容易被后续稳态执行摊薄。第三类则是运行时搬运相关成本，例如 memcpy_gc 以及后续在火焰图中持续可见的字符串替换、字符串拼接和 UTF-16 构造等路径，它们共同表明

表 5.2 DaCapo/pmd baseline 主要热点符号

符号	占比	类别
inflate_fast	3.53%	压缩/输入处理
INSTANCE_OF()	2.54%	解释器热点
SaveDestination::GetOrCreateFromMap<bool>()	2.51%	运行时固定成本
ExecuteSwitchImplCpp<false>()	2.43%	解释器热点
ElfDebugReader::ElfDebugReader(...)	2.24%	JIT 调试信息固定成本
memcpy_gc	1.92%	内存搬运
VisitFunctionSymbols(...)	1.83%	JIT 调试信息固定成本
RETURN_OBJECT()	1.45%	解释器热点

该 workload 的字符串构造、替换与拼接行为较为活跃。

因此, pmd 对本文具有两方面价值。一方面, 它能够敏感地反映解释器驻留时间是否被压缩; 另一方面, 它又能够观察运行时字符串热路径微优化是否真正命中热点。这使得 pmd 很适合作为执行引擎与运行时协同优化的主要观测样本。

5.2.2 SPECjvm2008 mpegaudio 特征

与 pmd 不同, SPECjvm2008/mpegaudio 更偏向经典 JVM 场景中的数值处理和数组访问。在方案筛选期保留的 baseline flamegraph 归档中, 该 workload 的热点组成已稳定体现出更强的数组、内存和分配器行为。其主要热点符号如表 5.3 所示。

表 5.3 SPECjvm2008/mpegaudio baseline 主要热点符号

符号	占比	类别
Builder::Ignore(...)	2.56%	运行时固定成本
DexFileVerifier::CheckIntraSection()	2.20%	运行时固定成本
ElfDebugReader::ElfDebugReader(...)	1.89%	JIT 调试信息固定成本
INSTANCE_OF()	1.82%	解释器热点
memset_gc	1.70%	内存搬运
RETURN_OBJECT()	1.31%	解释器热点
scudo::computeChecksum(...)	1.16%	分配器/固定成本
memcpy_gc	0.72%	内存搬运

从热点分布可以看出, mpegaudio 与 pmd 虽然都能观察到解释器和 JIT 调试

信息固定成本，但其收益响应边界并不相同。首先，mpegaudio 更依赖稳态代码质量，对单次运行中编译成本的摊薄能力要求更高。其次，它的热点中包含更多数组、内存和分配器相关行为，而不是以字符串处理为主。再次，mpegaudio 的单次 workload 较短，若某项优化只是更早触发编译，而不能同时显著提升稳态机器码质量，则很容易被额外的固定成本抵消。

为便于说明两类 workload 的差异，表 5.4 对它们的主要特征进行归纳。

表 5.4 pmd 与 mpegaudio 的特征对比

维度	pmd	mpegaudio
主要运行特征	解释器驻留明显，字符串处理频繁	浮点/数组处理较多，更关注稳态吞吐
主要瓶颈表现	解释器热点和字符串热路径并存	固定成本与稳态代码质量共同影响
对 JIT 策略的敏感性	对更早进入 baseline JIT 较敏感	对单纯提前编译不一定敏感
对运行时微优化的敏感性	对字符串热路径微优化较敏感	对字符串类优化响应较弱

正是由于这些 workload 在性能响应方向上存在明显差异，本文才能避免把某一项局部优化误写为普遍适用于所有场景的通用结论。在正式方案中，方案一主要命中 pmd 这类短生命周期负载，方案二主要用于验证字符串热路径微优化的收益边界，方案三则以 SPECjvm2008/scimark.lu.small 为主要受益对象；而 mpegaudio、sor.small 与 fft.small 则用于观察这些方案的跨 workload 可迁移性与副作用边界。

5.3 JIT 分层热度阈值调整（方案一）

5.3.1 问题背景与设计思路

ART 的 JIT 执行路径本质上属于分层优化系统：方法首先需要达到 warmup 阈值进入 baseline JIT，随后在热点继续积累时再进入优化重编译阶段 [6, 17]。在 LoongArch64 debug 构建下，本文观察到 pmd 这类短生命周期 workload 存在一个突出问题：许多热点方法在执行结束前仍然停留在解释器中，或仅在较晚阶段才进入 baseline JIT，从而导致解释器成本难以被及时回收。

基于这一观察，本文提出 JIT 分层热度阈值调整方案。其核心思想不是同时改写两层 JIT 的整体策略，而是只降低第一层 baseline JIT 的进入阈值，使 pmd 这类工作负载更早脱离解释器。与此同时，优化编译阈值 `optimize_threshold` 保持不变，以避免放大第二层重编译的全局副作用。换言之，该方案的目标是解决“第一阶段进入得太晚”的问题，而不是直接改变“第二阶段编译得是否足够激进”的问题。

从已有 Android JIT 研究看，移动端运行时的收益往往取决于“何时编译”和“编译成本如何摊薄”的匹配关系，而不是单纯追求更激进的优化等级；这一认识也与 hybrid JIT 和 ShareJIT 等工作的经验是一致的 [24, 37]。

为控制改动边界，本文对该方案设置了严格门控条件：仅在 LoongArch64、debug 构建、非 Java-debuggable 进程，且用户未显式传入 `JITWarmupThreshold` 参数时，才自动下调默认 warmup 阈值。正式方案最终采用 8192 作为目标值；中途也曾尝试过 16384，但未能保住 pmd 收益，因此未作为正式实现保留。

5.3.2 实现方法

该方案对应的正式实现版本下文简称为“方案一正式实现版本”。从代码结构上看，它主要分布在 `art/runtime/jit/` 下的三个文件中，具体包括：

1. 在 `jit_options.h` 中增加 `HasExplicitWarmupThreshold()`、`SetWarmupThreshold()` 以及显式阈值状态位，用于区分“默认值”和“用户手工配置值”。
2. 在 `jit_options.cc` 中记录 `JITWarmupThreshold` 是否由运行时参数显式传入，为后续门控提供依据。
3. 在 `jit.cc` 中新增 LoongArch64 分层阈值调节辅助逻辑，并在 `Jit::Create()` 阶段完成 debug 场景下的 warmup 阈值调整。

这种接入方式具有两点好处。首先，阈值修改发生在 JIT 创建阶段，不需要在解释器热路径中增加新的判定分支，因此不会引入额外的运行时检查成本。其次，通过“仅在未显式配置时才覆盖默认值”的策略，该方案避免了破坏开发者或测试脚本原有的命令行语义，使其更接近“平台默认策略微调”而非“强行接管所有 JIT 参数”。

5.3.3 实验结果与分析

在进入 benchmark 之前，本文首先对该方案进行了最小相关目标机回归，所执行的 708-jit-cache-churn、667-jit-jni-stub 等用例均通过，137-cfi 为自动跳过，不影响结论。随后，本文在 2026-03-13 同日 final-comparison 口径下采集正式 benchmark 结果，如表 5.5 所示。

表 5.5 JIT 分层热度阈值调整方案结果

Benchmark	Baseline	方案结果	相对变化
DaCapo/pmd	3272 msec	3164 msec	+3.30%
SPECjvm2008/mpegaudio	25.38 ops/m	25.74 ops/m	+1.42%
SPECjvm2008/scimark.lu.small	46.22 ops/m	62.05 ops/m	+34.25%

从正式单次结果看，该方案在 pmd 上仍是最直接、最容易解释的正收益样本，而 mpegaudio 上仅呈现轻微正向波动。lu.small 上出现的 +34.25% 更适合作为正式单次结果中的附带观察，而不宜单独上升为已经稳定复现的论文主结论。结合热点与 workload 生命周期，可以得到三点解释。

第一，对 pmd 而言，更早进入 baseline JIT 确实压缩了解释器驻留时间。与 baseline 相比，INSTANCE_OF + ExecuteSwitchImplCpp + RETURN_OBJECT 相关解释器热点占比由约 6.4% 下降到约 5.2%，ElfDebugReader + VisitFunctionSymbols 相关固定成本也由约 4.07% 下降到约 3.75%。这说明该方案虽然没有直接优化解释器代码本身，但通过更早触发 baseline JIT，使 pmd 中部分热点方法提前脱离了解释执行阶段。

第二，对 lu.small 而言，该方案并未改变向量化能力本身，但在正式单次结果中仍伴随 +34.25% 的提升。一个可能的解释是，更早进入 baseline JIT 使得数值循环也提前获得了编译执行机会，从而间接改善了 lu.small 的吞吐表现。不过，这更适合作为附带观察，而不是已经经由重复测量确认的独立主张。换言之，方案一并不只服务于字符串型或解释器型 workload，但它对数值型 benchmark 的影响在当前材料中仍应保持谨慎表述。

第三，对 mpegaudio 而言，+1.42% 的提升幅度仍然较小，不足以单独构成论文主结论。mpegaudio 更依赖稳态代码质量，而不是单纯减少少量解释器时间；因此该方案更适合被表述为“对短生命周期、解释器占比较高或 JIT 进入偏晚的

workload 有利”，而不应被概括为“通用 JIT 优化”。

此外，16384 的调参尝试在 pmd 上得到 3602 msec 的结果，明显劣于正式方案。这一现象表明，JIT 热度阈值并非单调递减或递增的简单变量，而是与 benchmark 生命周期、固定成本摊薄能力以及 LoongArch64 debug 构建的整体执行特征密切相关。不过，本文在后文的同日重复测量验证中也会看到，该方案在 pmd 上虽然存在正收益样本，但方差仍然明显高于 baseline，因此其最稳妥的结论应是“方向正确，但稳定性仍需更严格控制实验进一步确认”。

5.4 运行时字符串搬运批量化（方案二）

5.4.1 问题背景与设计思路

在对 pmd 的火焰图和日志持续分析后，本文发现运行时字符串相关路径是另一类稳定热点。具体而言，字符串替换、字符串拼接和 UTF-16 构造三类运行时路径在多轮实验中反复出现。这说明对于 pmd 这类频繁构造短字符串、执行替换和拼接操作的 workload，运行时字符串搬运成本已经足以影响最终分数。已有 JVM 字符串优化研究同样指出，字符串表示、复制和编码转换路径中的局部固定成本，会在短字符串密集 workload 中被持续放大 [25]。

基于这一观察，本文提出运行时字符串搬运批量化方案。其基本思路是：在不改变字符串压缩判定、不破坏语义边界的前提下，将若干高频字符复制路径从逐元素循环改为固定宽度展开实现，从而降低循环控制开销与分支密度。相比直接接入更激进的体系结构特化指令，这一方案仍然属于保守微优化，其目标不是重写整个字符串实现，而是针对正式 workload 中已经确认的热路径做局部减负。

5.4.2 实现方法

该方案对应的正式实现版本下文简称为“方案二正式实现版本”，主要涉及字符串分配内联实现、字符串核心实现以及一个目标机回归测试文件。其实现可以概括为三部分。

1. 在 string-alloc-inl.h 中新增一组字符串搬运辅助函数，分别覆盖字节复制、UTF-16 与字节互转、UTF-16 字节互转以及字符替换等场景，统一承载 8 路展开后的批量复制逻辑。

2. 在 string.cc 中将字符串替换、字符串拼接和 UTF-16 构造三条热路径接入上述 helper，使字符串构造、拼接和替换尽量走同一套低开销搬运逻辑。

3. 在 `art/test/624-checker-stringops/src/Main.java` 中新增小长度 UTF-16 构造回归，用于覆盖 `length < 8` 的边界情况。

值得强调的是，该方案在实现过程中曾暴露出一个真实边界缺陷。初版代码使用 `for (; i <= length - 8; ...)` 作为展开循环条件，而 `length` 的类型为 `uint32_t`；当字符串长度小于 8 时，`length - 8` 会发生无符号下溢，进而导致越界访问，并在 `pmd` 上表现为 `exit=139`。最终本文将条件修正为 `for (; i + 8 <= length; i += 8)`，并补充了小字符串测试覆盖该路径。这一修复过程也再次说明，运行时微优化虽然看似局部，但若边界处理不严谨，其风险并不低于编译器策略调整。

5.4.3 实验结果与分析

在验证环节，该方案通过了 624-checker-stringops、729-checker-polymorphic-intrinsic、595-profile-saving、570-checker-osr、667-jit-jni-stub 和 708-jit-cache-churn 等最小相关测试。正式 benchmark 结果如表 5.6 所示。

表 5.6 运行时字符串搬运批量化方案结果

Benchmark	Baseline	方案结果	相对变化
DaCapo/pmd	3272 msec	3244 msec	+0.86%
SPECjvm2008/mpegaudio	25.38 ops/m	25.56 ops/m	+0.71%
SPECjvm2008/scimark.lu.small	46.22 ops/m	42.11 ops/m	-8.89%

与方案一、方案三相比，方案二的收益更弱，也更依赖 workload 类型。首先，在 `pmd` 与 `mpegaudio` 上，它都只表现出约 1% 量级的轻微正收益。这说明字符串搬运固定成本确实可以被压低，但它不是这些 workload 的主导瓶颈，因而最终总收益只能停留在弱正区间。

其次，该方案在 `lu.small` 上出现 -8.89% 的明显回退，这恰好从反面说明它命中的并非数值内核主路径。对于以连续浮点数组计算为主的 workload，字符串 helper 的开销即便存在，也不足以覆盖额外代码体积、调度变化和潜在缓存扰动带来的副作用。因此，方案二只能被界定为“字符串密集场景的定向微优化”，而不应把它写成通用默认策略。

另外，当前脚本日志中提示 `perf-*.map` 缺失，导致部分 JIT Java 符号并不完整。因此，本文不会将该方案写成“火焰图显示字符串热点显著消失”，而是采用更谨慎的表述：字符串热路径在 `profile` 中持续可见，但在正式单次对比中 `pmd`

与 mpegaudio 均未出现系统性回退，这足以说明运行时字符串搬运批量化至少降低了部分热点路径的固定成本；只是这种降低幅度还不足以在更广的 workload 上兑现为稳定大收益。

5.5 LoongArch64 LSX 浮点 SIMD 向量化（方案三）

5.5.1 问题背景与设计思路

前两项方案主要围绕解释器驻留和运行时固定成本展开，而对 Scimark 的 lu.small 这类规则浮点数值内核而言，更核心的瓶颈在于：LoongArch64 的 ART optimizing 编译器尚未提供最小可用的浮点 SIMD 落地路径，导致 loop vectorization 即便在前端识别出可向量化循环，也难以在后端真正生成 LSX 向量代码。

基于这一观察，本文提出 LoongArch64 LSX 浮点 SIMD 向量化方案。该方案的目标并不是一次性补齐完整的 LoongArch64 SIMD 后端，而是在严格收缩正确性边界的前提下，为 float32/float64 浮点向量提供一个最小可用闭环，使规则 dense linear algebra 内核至少能够完成“标量广播、向量 load/store、向量加减乘”这一组最基本的 lowering。对 scimark.lu.small 而言，这已经足以覆盖其最核心的 rank-1 update / DAXPY 型热循环。

5.5.2 实现方法

该方案对应的正式实现版本下文简称为“方案三最小 SIMD 正式版本”。从代码结构上看，它主要分布在 art/compiler/optimizing/ 下的三个文件中，具体包括：

1. 在 loop_optimization.cc 中为 InstructionSet::kLoongarch64 打开受限的浮点向量化入口，允许 float32/float64 在无 if-cond、无 div、无 reduction 的前提下进入向量化流程。
2. 在 code_generator_loongarch64.h 中将 SIMD 图下的寄存器宽度与保存恢复宽度统一设置为 16 bytes，为 LSX 128-bit 浮点向量寄存器分配建立基础约束。
3. 在 code_generator_loongarch64.cc 中补齐最小 LSX 指令编码与相关 HIR lowering，包括 HVecReplicateScalar、HVecLoad、HVecStore、HVecAdd、HVecSub 和 HVecMul，并打通 SIMD 图下的寄存器保存恢复与栈槽搬运。

这种“最小能力子集”设计的价值在于，它只针对正式 benchmark 真正需要

的那一小块向量语义开口，避免为了尚未验证的复杂模式引入额外错误面。与此前失败的“标量循环展开增强方案”相比，本方案不是单纯增加标量代码体积，而是直接把浮点数值内核从标量乘减流改为 128-bit LSX 向量流，因此具有更明确的吞吐改进预期。

5.5.3 实验结果与分析

在进入 benchmark 之前，本文首先执行了与 SIMD 后端最直接相关的最小相关目标机回归，527-checker-array-access-simd、656-checker-simd-opt、570-checker-osr、570-checker-osr-locals 和 708-jit-cache-churn 均通过。随后，在 2026-03-13 同日 final-comparison 口径下采集正式 benchmark 结果，如表 5.7 所示。

表 5.7 LoongArch64 LSX 浮点 SIMD 向量化方案结果

Benchmark	Baseline	方案结果	相对变化
DaCapo/pmd	3272 msec	3270 msec	+0.06%
SPECjvm2008/mpegaudio	25.38 ops/m	25.84 ops/m	+1.81%
SPECjvm2008/scimark.lu.small	46.22 ops/m	66.74 ops/m	+44.40%
SPECjvm2008/scimark.fft.small	76.43 ops/m	74.44 ops/m	-2.60%

从结果看，方案三是本文当前最明确、最容易解释的正收益方案。lu.small 获得 +44.40% 的单方案收益，说明在 LoongArch64 原本缺失浮点 SIMD lowering 的前提下，只要补齐最小可用的 LSX 浮点向量路径，规则 dense linear algebra 内核就能够显著受益。

与此同时，pmd 基本持平、mpegaudio 只有轻微波动，说明该方案不会自然改善字符串型或控制流型 workload，它的收益边界主要集中在规则浮点循环。更有代表性的是，fft.small 仍然出现了轻微回退，而同轮补充的 sor.small 也仅接近 1.0；这说明“数值 workload”并不是一个足够精细的分类，是否真正获益仍取决于向量模式、算子种类与后端支持能力是否匹配。

同时需要指出的是，当前方案三仍然是一个面向论文实验的最小能力子集：它默认目标设备具备 LSX，并且仅覆盖 float32/float64 的基本向量 load/store、标量广播与加减乘 lowering。因此，本文对方案三的结论应理解为“最小可用的 LoongArch64 LSX 浮点向量路径已经足以在规则数值内核上兑现显著收益”，而不是“LoongArch64 的完整 SIMD 后端已经在 ART 中全面成熟”。

5.5.4 完整 SIMD 覆盖版的后续消融

在完成正式方案三之后，本文又围绕 LoongArch64 SIMD 能力补齐做了一轮后续候选实验，比较三种状态：S0 为无 SIMD 基线，S1 为前文已采用的最小可用 SIMD 版本，S2 为后续整理出的“完整 SIMD 覆盖版”。该轮实验使用 2026-03-18 独立采集的数据集，对当前设备上全部可稳定运行的 21 个 SPECjvm2008 workload 和 DaCapo 的 avrora、pmd、h2 进行比较；其中 h2 仅在 JIT 模式可用，全部 workload 默认重复 3 轮，lu.small、sor.small、fft.small、mpegaudio、h2 和 pmd 在 JIT 模式下扩展到 5 轮。实验的目的不是替换前文正式方案三的结论，而是回答“能力覆盖继续扩大后，收益是否还能保持整体上升”这一问题。

需要额外说明的是，这一 S0/S1/S2 消融与表 5.7 并不共享同一条正式 baseline 链。前文正式方案三使用的是 2026-03-13 的 final-comparison 数据集，对照对象为正式基线版本与方案三最小 SIMD 正式版本；而本节候选消融使用的是独立采集的数据集，三种状态分别锚定在关闭 Intrinsic 的候选基线、候选链最小 SIMD 版本和完整 SIMD 覆盖候选版上。因此，本节结果只用于比较“最小 SIMD”和“完整 SIMD 覆盖”之间的相对关系，而不与表 5.7 的绝对增益数值直接混算。

表 5.8 完整 SIMD 覆盖候选版的几何平均 speedup

比较	JIT 全部	JIT 关键 workload	JIT DaCapo	Interpreter 全部
S1 / S0	1.0318x	1.0453x	0.9884x	0.9994x
S2 / S0	1.0252x	1.0617x	1.0282x	0.9954x
S2 / S1	0.9936x	1.0157x	1.0402x	0.9960x

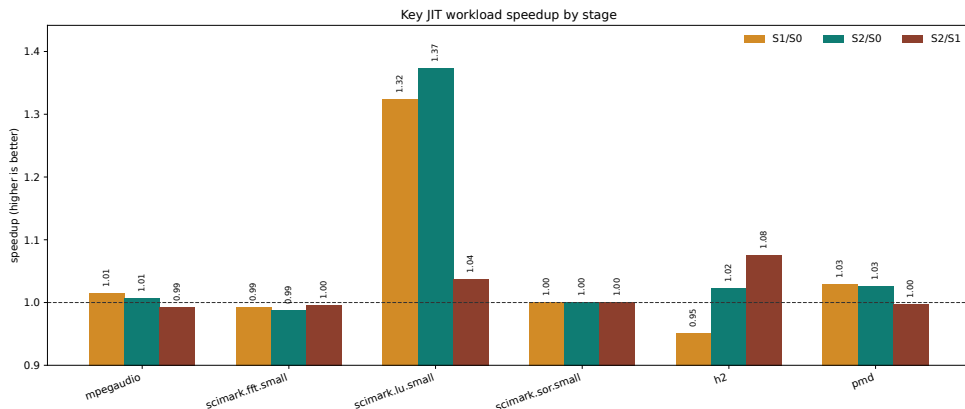


图 5.1 完整 SIMD 覆盖候选版在关键 workload 上的相对收益

从表 5.8 可以先得到一个重要结论：若以“全部 JIT workload 的几何平均”作为判断标准，则前文的最小可用版本 S1 仍然是当前更稳妥的正式方案，因为它相对 S0 的整体收益为 1.0318x，而 S2 相对 S0 为 1.0252x，S2 / S1 仅为 0.9936x。这说明继续扩大 SIMD 覆盖并不会自动把整体均值继续推高，部分 startup 或非关键 workload 上的小幅副作用已经足以抵消一部分新增收益。

但从图 5.1 和关键 workload 几何平均看，S2 也并非简单退化版本。相对 S1，S2 在关键 workload 上仍有 1.0157x 的几何平均增益，并在 DaCapo 子集上达到 1.0402x。最具代表性的样本包括：scimark.lu.small 的 S2 / S1 = 1.0371x，h2 的 1.0758x，以及 avrora 的 1.0497x。相对地，mpegaudio、scimark.fft.small 等 workload 没有继续提升，说明“完整覆盖”带来的收益主要集中在真正命中新 lowering 或新 idiom 的工作负载上，而不是对所有 benchmark 都形成同步抬升。

与此同时，interpreter 侧三组比较都基本贴近 1.0x。这说明本轮差异仍主要来自 JIT SIMD 路径，而不是设备漂移、部署差异或其它与向量化无关的实验扰动。因此，对于完整 SIMD 覆盖版，更准确的论文表述应是：“它补齐了更多 LoongArch64 SIMD 能力，并在关键 workload 上继续受益，但当前尚不足以在全量几何平均意义上替代最小可用 SIMD 版本成为新的正式默认方案。”这一点也从反面说明，后续若要继续扩大 SIMD 的默认收益，仅靠继续补 lowering 还不够，还需要结合更高层的循环模式选择、idiom 识别和 pass 级门控进一步控制副作用。

5.6 三项正式方案的统一对比与重复测量验证

5.6.1 同日正式单次结果总览

为了在统一口径下观察三项正式方案的收益边界，本文将 2026-03-13 同日 baseline 与三项单方案结果汇总为图 5.2。图中同时保留 pmd、mpegaudio、lu.small、sor.small 和 fft.small 五个 workload，其中 pmd 主要观察 JIT 时机与运行时路径，后三者主要观察数值型 workload 对 SIMD 方案的响应边界。

从图 5.2 可以归纳出四点。第一，方案一在 pmd 上最具代表性，同时在正式单次结果中对 lu.small 也出现明显附带提升。第二，方案二在 pmd 与 mpegaudio 上只有弱正收益，却在 lu.small 上明显回退，说明它只能作为字符串密集 workload 的局部优化。第三，方案三在 lu.small 上形成压倒性的单方案收益，但对 pmd 近

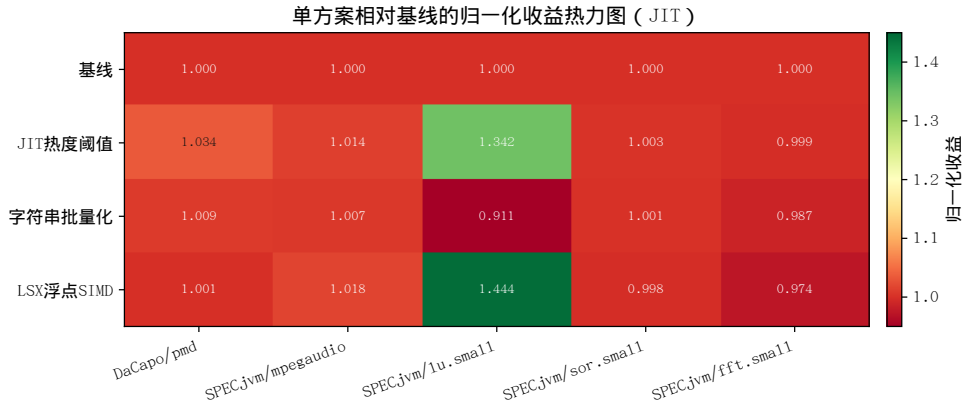


图 5.2 三项正式方案相对同日基线的归一化收益热力图

乎中性，说明该 SIMD 方案不会自然改善非数值主导 workload。第四，sor.small 与 fft.small 都没有复现 lu.small 的大幅收益，这进一步说明并非所有数值循环都会自动从当前最小 LSX lowering 中受益。

5.6.2 同日重复测量验证

单次正式结果能够反映方案在统一设备状态下的直接收益，但仍不足以回答“该收益是否能在相邻时间窗口中稳定复现”。为避免把不同日期的设备状态波动误写成方案收益，本文又在 2026-03-14 单独建立了一个重复测量数据集，只验证两条最重要的性能主张：方案一在 DaCapo/pmd 上的收益，以及方案三在 lu.small 上的收益。考虑到样本数仅为 3 轮，本文不强行使用显著性术语，而采用图 5.3 直接展示原始散点与均值。

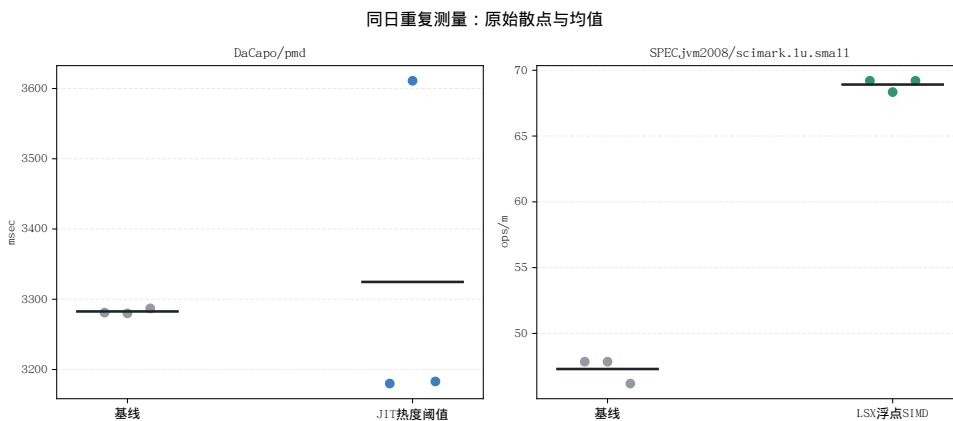


图 5.3 同日重复测量下方案一与方案三的原始散点和均值

从图 5.3 可以看出，方案三在 lu.small 上的三轮结果均落在 68–69 ops/m 区间，相对 baseline 的均值提升达到 +45.70%，标准差仅 0.50 ops/m，属于可直接

写入论文主结论的稳定收益。相反，方案一在 pmd 上虽然有两轮结果落在 3180 和 3183 msec，与正式单次对比的正收益方向一致，但第三轮出现 3611 msec 的高延迟异常值，导致 3 轮均值回落到 -1.26%。因此，对方案一更稳妥的表述应是：它在 pmd 上存在可复现的正收益样本，但当前仍未达到“均值稳定正收益”的强结论门槛。

5.6.3 三项方案的叠加并不天然成立

除单方案比较外，本文还对三项方案的叠加状态进行了额外整理。结果表明，即使各单方案都围绕正式 workload 中的真实热点展开，三者也并不天然具备“统一默认启用后仍可同时受益”的可组合性，如表 5.9 所示。

表 5.9 方案叠加状态与单方案结果对照

状态	pmd	mpegaudio	lu.small
方案一单独启用	+3.30%	+1.42%	+34.25%
方案一 + 方案二	-8.56%	+1.65%	+29.16%
方案一 + 方案二 + 方案三	-9.54%	-12.45%	+50.41%

从表 5.9 可以看到，方案二加入后，pmd 立即从方案一单独启用时的正收益翻转为明显回退；进一步叠加方案三后，lu.small 继续上升，但 mpegaudio 又出现了显著负向波动。这说明执行引擎优化并不是将所有单独有效的局部策略同时启用就能得到更好结果。不同方案可能命中不同 workload 的主路径，也可能在代码体积、运行时固定成本和 JIT 行为层面产生相互干扰。因此，本文保留三项方案的方式是“作为独立优化方向分别分析和验证”，而不是将其直接写成可统一叠加的默认配置。

5.7 Intrinsic 机制的补充验证

前三项正式方案分别面向 JIT 进入时机、运行时字符串搬运和 LSX 浮点向量 lowering，但它们并不能直接回答另一个基础问题：已经补齐的 LoongArch64 Intrinsic 机制本身，究竟对总体 benchmark 带来多大收益。由于 Intrinsic 是新架构 ART 执行引擎成熟度的重要边界，本文额外设计了一组“关闭/开启 Intrinsic”的消融实验，用于量化该机制在当前 LoongArch64 ART 中的实际贡献。这里关闭的不是整个 JIT，而是 LoongArch64 optimizing 路径中的 invoke intrinsic dispatch；

因此，该实验主要测量的是 HInvoke Intrinsic 在 JIT/optimizing 路径上的收益，而解释器模式则被用作负对照。

该补充实验仍沿用第 4 章的统一验证链路和目标设备，比较对象分别为“关闭 LoongArch64 invoke Intrinsic dispatch”的基线版本与“恢复 Intrinsic 支持”的对照版本。benchmark 选择当前目标设备上稳定可用的全部 workload：SPECjvm2008 的 21 个子项和 DaCapo 的 avrora、pmd、h2（其中 h2 仅在 JIT 模式可用），并分别在 interpreter 与 JIT 模式下各重复 3 轮。对于 SPECjvm2008，本文采用 enabled_ops / disabled_ops 计算 speedup；对于 DaCapo，采用 disabled_time / enabled_time 计算 speedup，因此表中数值越大表示开启 Intrinsic 后越有利。

表 5.10 Intrinsic 开关消融实验的几何平均 speedup

模式	SPEC startup	SPEC throughput	DaCapo	总体
interpreter	1.0189x	0.9980x	1.0359x	1.0112x
JIT	1.0339x	1.0508x	1.0387x	1.0415x

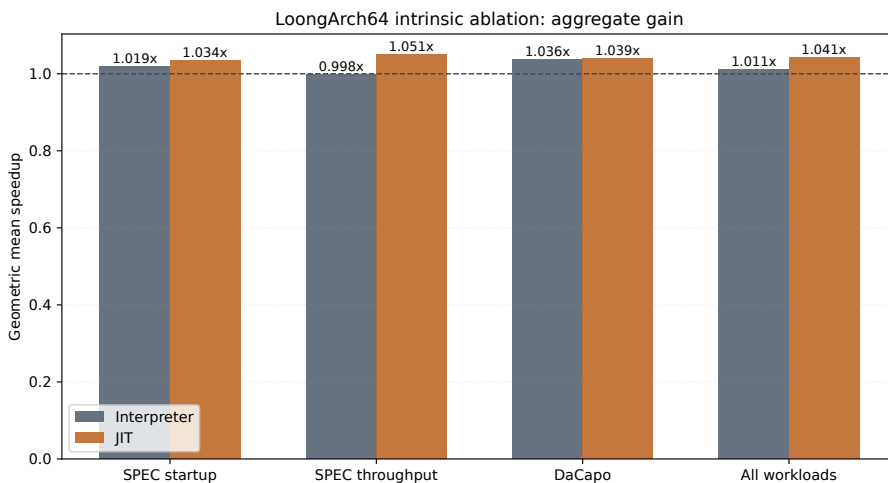


图 5.4 LoongArch64 Intrinsic 开关消融实验的几何平均 speedup

从表 5.10 和图 5.4 可以看到，Intrinsic 的主要收益确实集中在 JIT 路径。JIT 模式下，全部可比较 workload 的几何平均 speedup 为 1.0415x，其中 SPEC throughput 达到 1.0508x；而 interpreter 模式总体仅为 1.0112x，23 个可比较 workload 中有 19 个落在 $\pm 2\%$ 区间内，更接近设备抖动和短时噪声。这一结果与 Intrinsic 的实现位置是一致的：当前 LoongArch64 Intrinsic dispatch 位于 optimizing 编译器而非解释器主体，因此其收益主要通过已编译热点代码兑现。

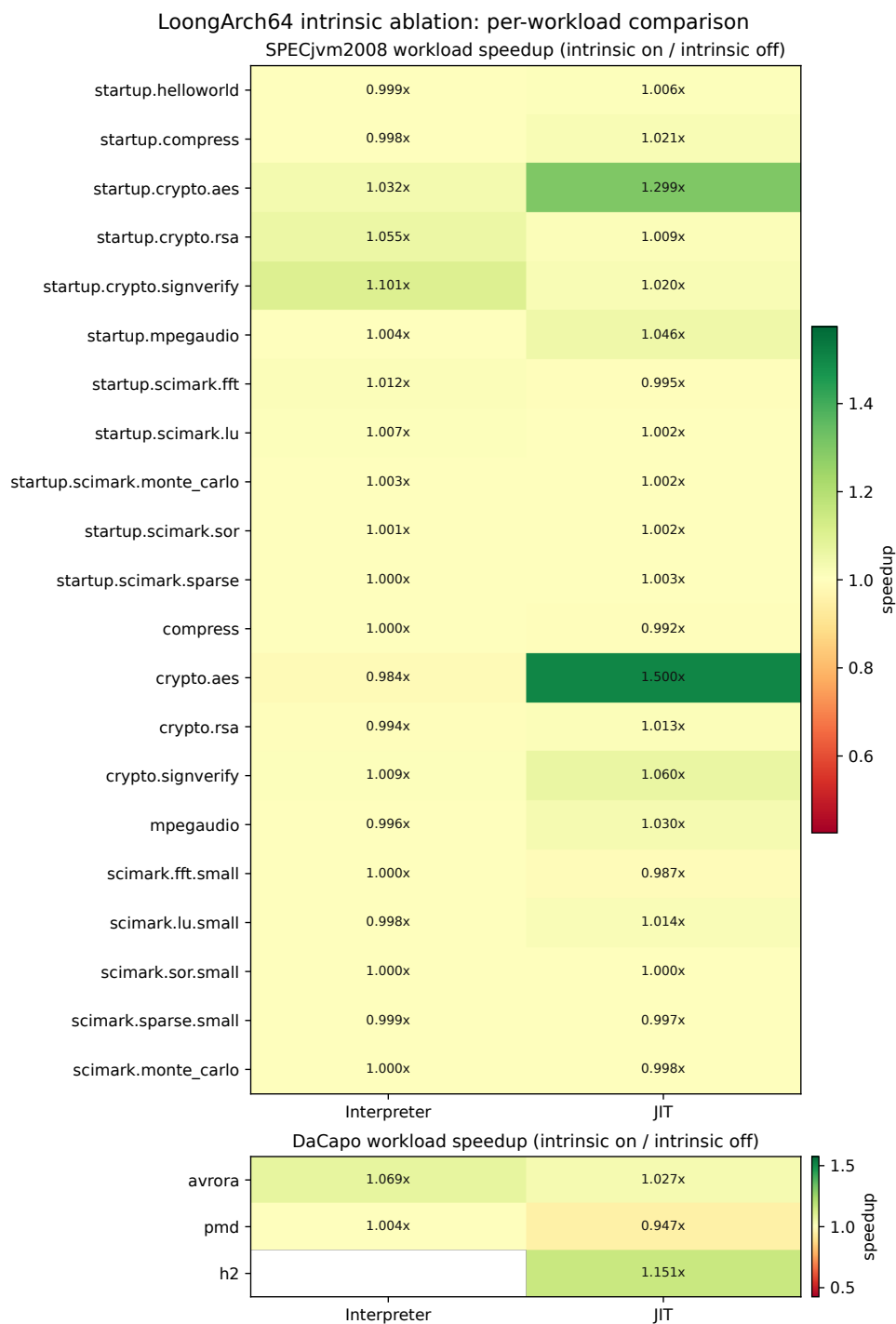


图 5.5 LoongArch64 Intrinsic 开关消融实验的逐 workload 收益热力图

从逐 workload 结果看，图 5.5 进一步展示出 JIT 下最具代表性的收益点是 crypto.aes (1.4996x)、startup.crypto.aes (1.2993x)、h2 (1.1513x) 和 crypto.signverify (1.0604x)。这些 workload 的共同特点是存在可枚举的标准库热点语义，能够被 Intrinsic 直接映射为更短的机器码序列或更直接的辅助路径。相对地，pmd 仍然出现了约 -5.26% 的回退，说明 Intrinsic 并不是对所有 Java workload 都普遍有效：如果主导瓶颈仍在 JIT 时机、字符串运行时路径或未命中的库热点上，则单独打开 Intrinsic 并不能保证端到端收益。

这组补充实验对全文有两点意义。第一，它从实验上证明了“补齐 Intrinsic 支持”确实不是单纯的成熟度修饰，而是能够在 JIT 主路径上形成可量化收益的执行引擎能力。第二，它也帮助区分了 Intrinsic 与方案三 SIMD 向量化的收益边界：前者主要改善标准库和内建语义路径，后者主要改善显式规则数值循环；二者命中的热点类型不同，因此在论文中应分别论证，而不宜混写为同一类后端优化。

5.8 失败方案对照分析

5.8.1 解释器与 JIT 热路径类失败方案

为了说明方案一为何选择“仅降低 warmup 阈值”而不是沿其它 JIT 方向继续放大改动，本文选取两项与其技术路径最接近的失败方案进行对照，如表 5.11 所示。

表 5.11 与 JIT 热路径相关的失败方案对照

方案	pmd	mpegaudio	主要失败原因
JIT 优化重编译阈值调整	-11.23%	-2.03%	只加快第二层优化编译，但第一层 baseline JIT 进入仍然过晚，收益难以兑现
OSR 热路径内联策略调整	-17.78%	-2.34%	节省的只是函数调用边界，未命中正式 JIT 模式下的主导瓶颈，且可能引入代码膨胀

第一项失败方案是“JIT 优化重编译阈值调整”。该方案试图只降低 optimize_threshold，希望让 mpegaudio 这类更依赖稳态代码质量的 workload 更早进入优化重编译阶段。然而结果表明，pmd 下降 11.23%，mpegaudio 也下降 2.03%。

其根本原因在于：在当前 baseline 下，很多方法连第一层 baseline JIT 都进入得不够早，更谈不上及时享受第二层优化机器码的收益。这一失败样本恰好从反面证明了正式方案一的合理性，即当前更关键的问题是“第一层进入过晚”，而不是“第二层不够激进”。

第二项失败方案是“OSR 热路径内联策略调整”。它尝试将 OSR 判定逻辑直接内联到解释器分支热路径中，减少回边上的函数调用与参数传递开销。但正式 benchmark 口径固定为 JIT 模式，此时运行时本就存在 Jit 对象，OSR 内部判定逻辑仍然要继续执行，因此真正被省掉的工作量有限。相反，由于分支处理热路径被扩张，反而可能增加指令缓存压力，最终导致 pmd 明显回退。该失败案例说明，对于解释器与 JIT 交界处的热点，单纯做调用边界层面的微调并不一定能够换来整体收益。从反面看，这也说明 quickening、inline cache 一类解释器优化只有在正式 workload 的主导瓶颈确实位于解释器分派或分支预测路径时，才更可能兑现为整体性能收益 [22, 23]。

5.8.2 运行时微优化类失败方案

与正式方案二对应，本文还尝试过两项更偏“通用内存搬运”的微优化方案，但它们都未形成收益，如表 5.12 所示。

表 5.12 与运行时搬运相关的失败方案对照

方案	pmd	mpegaudio	主要失败原因
原始数组批量搬运展开	-9.79%	-1.72%	命中的只是次级热点，循环展开收益不足以覆盖额外代码体积与调度变化
LoongArch64 memcpy 快速入口接线	-11.95%	-2.45%	仅补齐 quick entrypoint，未证明 LoongArch64 热路径真正稳定发射到该入口

第一项失败方案是“原始数组批量搬运展开”。该方案在 PrimitiveArray 的前向与后向复制路径中做 4 路展开，试图降低小到中等长度数组搬运时的循环控制开销。从火焰图看，float 和 double 两类原始数组复制符号确实存在，但它们在正式 baseline 中始终不占主导地位；真正决定总分的，仍然是解释器热点与 JIT 调试信息固定成本。因此，哪怕数组搬运 helper 的局部效率有所提高，也很

难转化为整体 benchmark 收益。

第二项失败方案是“LoongArch64 memcpy 快速入口接线”。该方案的出发点是参考其它架构，为 LoongArch64 补齐 `qpoints->SetMemcpy(memcpy)`，希望字符串、数组和运行时搬运路径能够共享更优的快速入口。然而进一步检索发现，当前仓库中与 `kQuickMemcpy` 或 `pMemcpy` 直接相关的使用点非常有限，并没有证据表明 LoongArch64 的 JIT 后端或主要运行时热路径会在正式 benchmark 中稳定发射到这一入口。因此，该方案虽然“补齐了接口”，却没有真正改变热点路径的执行方式，最终也未能带来收益。

为了从整体上观察失败方案的分布边界，图 5.6 汇总了当前已归档失败方案相对于固定 baseline 的归一化结果。其中，DaCapo/pmd 按 $\text{baseline_time} / \text{current_time}$ 计算，SPECjvm2008/mpegaudio 按 $\text{current_ops} / \text{baseline_ops}$ 计算，因此图中 1.0 表示与 baseline 持平，低于 1.0 表示回退，灰色 NA 表示该方案未扩展到对应 benchmark。为提高图面可读性，纵轴标签直接采用“机制名”而非内部编号式代号。可以看到，绝大多数失败方案即便触达了某个局部热点，也未能跨过 benchmark 级总收益门槛。

5.8.3 失败方案的共性原因

综合以上失败样本，可以将其共性原因概括为以下三点。

1. **命中的并非主导热点。**无论是数组搬运局部展开，还是单纯补齐 `memcpy quick entrypoint`，它们都触达了“存在但不占主导”的次级热点，因此难以改变最终 benchmark 分数。

2. **放大了全局副作用，却没有换来等价收益。**例如只下调 `optimize_threshold` 会增加第二层编译时机，但在第一层还未及时进入时，这种全局编译行为放大只会增加额外成本。

3. **方案原理与正式 workload 的生命周期不完全匹配。**一些看似合理的优化，例如 OSR 热路径内联或 `memcpy` 入口补齐，在特定模式下可能有效，但与当前固定的 JIT、debug、短生命周期 benchmark 口径并不完全一致，因此难以在正式结论中站得住脚。

这些失败案例说明，LoongArch 平台 ART 的优化并不是缺乏系统规划的局部修补过程，而是一个必须持续围绕 workload 主热点、执行模式和固定成本展开的筛选过程。正式方案之所以能够保留，并不只是因为它们“跑分更高”，更

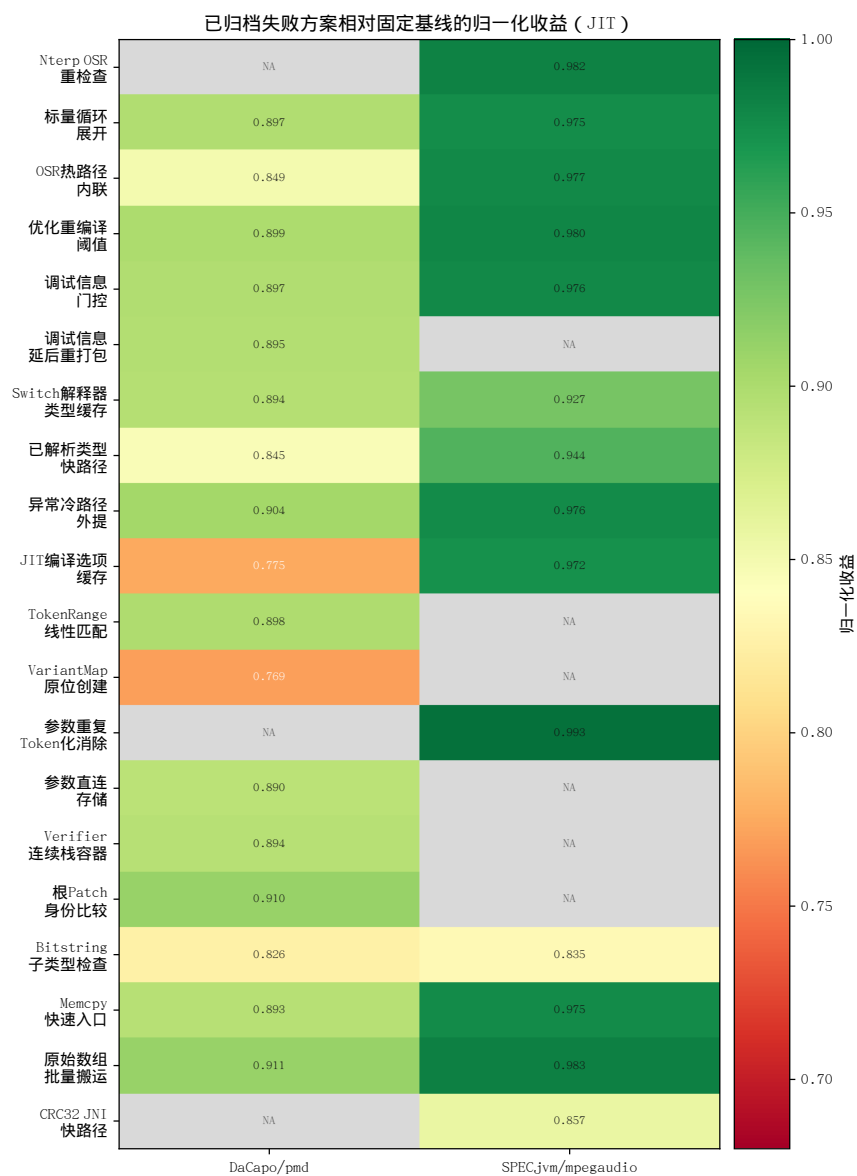


图 5.6 已归档失败方案相对固定 baseline 的归一化收益热力图

重要的是它们在热点定位、风险控制和收益兑现之间形成了更稳定的闭环。

5.9 正收益方案与失败方案的对比总结

5.9.1 可门控的局部调整与最小能力补齐更易兑现收益

从三项正式方案和四项失败方案的对比可以看出，真正能够形成收益的改动并不一定最“宏大”，但通常都满足两个条件：其一，作用范围清晰、门控条件明确；其二，新增能力严格收敛在正式 workload 确实需要的那一小部分。方案一只在特定平台与构建口径下下调 baseline warmup 阈值，方案二只在字符串热路径上做局部批量化实现，方案三也没有试图一次性补齐完整 SIMD 后端，而是仅为正式 benchmark 需要的 float32/float64 向量加减乘与 load/store 打开最小可用闭环。相反，失败方案往往存在两类问题：要么只触达了并不主导的辅助路径，要么改变了更大范围的运行时行为，却没有足够证据证明这些全局变化可以被 benchmark 生命周期摊薄。

5.9.2 不同 workload 的收益边界

本文的一个重要结论是：同一方案在五个 workload 上的响应分布并不天然一致。方案一在 pmd 的正式单次结果中最具代表性，对 lu.small 也有明显附带收益，但重复测量显示其在 pmd 上仍存在较大方差，因此更适合写成“方向正确但稳定性不足”。方案二在 pmd 与 mpegaudio 上只有弱正收益，却在 lu.small 上明显回退，说明它只适合作为字符串密集场景的定向微优化。方案三则在 lu.small 上形成稳定大幅收益，但对 pmd 基本中性，且未在 sor.small、fft.small 上复现同等级提升。对于系统优化研究而言，这一点尤为重要：论文中的“有效”应当建立在明确 workload 边界之上，而不是将局部收益简单外推为所有场景下的普遍规律。

5.9.3 面向 ART 执行引擎的优化方法论

结合本章的成功与失败样本，可以进一步提炼出面向 LoongArch 平台 ART 执行引擎的三条优化方法论。

1. 先区分筛选口径、正式对比口径与重复测量口径，再解释收益。对执行引擎这类系统软件而言，若把热点定位阶段的 baseline、正式单次比较和复测数据混为一谈，则任何跑分提升都难以解释其真实来源。

2. 优先补齐主导路径所需的最低可用能力，再讨论更大范围改写。本文中真正有效的方案，要么直接命中解释器驻留和字符串热路径，要么为规则浮点内

核补齐最小 LSX lowering；而多项失败方案之所以失败，恰恰是因为它们只触达了次级热点，或放大了全局行为。

3. **把失败样本与不稳定正收益一起当作筛选证据。**失败方案说明哪些方向在当前平台、构建口径和 workload 生命周期下并不成立；重复测量则说明哪些“看起来有效”的单次结果还不足以升格为论文主结论。两者共同约束，才能让最终结论更稳健。

5.10 本章小结

本章在固定 baseline 和统一验证链路下，围绕 JIT 分层热度阈值调整、运行时字符串搬运批量化以及 LoongArch64 LSX 浮点 SIMD 向量化三项正式方案，开展了系统的性能优化研究。正式单次对比表明，方案一在 pmd 上最具代表性，方案二只体现弱正收益且更偏定向微优化，方案三则在 lu.small 上形成最明确的单方案提升；同日重复测量进一步说明，方案三已经具备可直接写入论文主结论的稳定性，而方案一仍需以“存在正收益样本但方差较大”的保守口径表述。结合 JIT 热路径类失败方案和运行时微优化类失败方案的对照分析，本文说明了 LoongArch 平台 ART 在当前阶段更适合优先保留“命中主热点、门控边界清晰、收益能够在统一口径下复核”的优化方向，而非盲目扩大编译行为或在次级辅助路径上做局部修补。上述结论为后续总结全文贡献与展望后续工作奠定了基础。

第6章 总结与展望

6.1 本文工作总结

本文面向 AOSP 15 的 ART 运行时系统，围绕 LoongArch 平台执行引擎的功能可用、语义可验证、性能可优化三个目标开展研究。前文工作表明，对于一个尚无官方 ART 主线支持的新架构平台，真正的难点并不只是补齐若干体系结构相关文件，而是需要同时完成解释执行、JIT 编译、JNI 桥接、运行时入口点和验证评测链路的系统闭环。在此基础上，性能优化也不能依赖零散调参，而必须建立在统一 baseline、可复现实验流程和失败方案归档之上。

围绕这一主线，本文首先完成了 LoongArch 平台 ART 执行引擎关键模块的适配设计与实现，使 AOSP 15 上的 C++ 解释器、Nterp、JIT 编译器后端、JNI 编译器和若干运行时快速路径能够在目标平台形成完整执行闭环；其次，建立了覆盖编译、部署、target 回归、libcore 验证和 benchmark 评测的统一实验链路，使“移植完成”与“语义正确”之间不再停留于经验判断，而是具备可重复验证的依据；最后，在固定 JIT baseline 下，对典型 workload 的热点分布、收益边界、重复测量稳定性和失败原因进行了系统分析，形成了面向 LoongArch 平台 ART 的阶段性优化结论。

需要额外说明的是，附录中的 DexFileVerifier 阶段链实验属于与正文主线正交的启动期探索方向。该部分使用独立 baseline，用于补充观察加载期热点的局部优化边界，不并入第 5 章固定 baseline 下的主结论比较。

6.2 主要工作与贡献凝练

6.2.1 完成 LoongArch 平台 ART 执行引擎适配

本文以 AOSP 15 ART 为基础，围绕执行引擎相关核心模块，系统完成了 LoongArch 平台上的解释器、Nterp、JIT 编译器后端、JNI 编译器和运行时入口点等适配工作，构建了可运行、可验证的新平台 ART 执行环境。与早期版本 ART 或单纯 HotSpot/JVM 移植相比，AOSP 15 中执行引擎涉及的子系统更多、模块间耦合更紧，因此本文并未采用逐提交罗列方式描述实现，而是以执行路径为主线，对汇编器、解释器、JIT 与运行时辅助路径的配合关系进行了模块化梳理。

这一工作作为后续在国产处理器平台上继续推进 Android 运行时适配提供了较新的工程基线。该结论主要由第 3 章的适配设计与实现，以及第 4 章中三条主要执行路径的 target run-test 和 libcore 验证结果共同支撑。

6.2.2 建立系统化验证与实验方法

本文围绕移植与优化过程，建立了覆盖 Host 侧构建、Target 回归、libcore 测试与 benchmark 实验的验证方法，并统一了正式 baseline、日志归档与热点分析口径。相较于只给出仅能启动或少量样例通过验证的移植报告，本文更强调 correctness 与 performance 两条证据链的统一：一方面，通过目标机 run-test 与 libcore 用例约束语义正确性；另一方面，通过固定 benchmark 目录、保存 run.log、perf.data、report_symbol.txt 和火焰图等产物，保证性能分析具备可追溯性。该结论主要由第 4 章中统一验证链路、正式 baseline 口径和有效性威胁控制的组织方式支撑，该方法同样适用于后续新增方案的持续筛选。

6.2.3 获得面向典型 workload 的优化结论

第 5 章围绕固定 baseline 下的典型 benchmark，提出并验证了三项正式方案，并通过同日重复测量进一步约束结论强度。JIT 分层热度阈值调整方案在 DaCapo/pmd 的正式单次结果中最具代表性，且在 lu.small 上也出现明显附带提升，但该附带结果当前仍以正式单次观察为主；重复测量显示其在 pmc 上方差仍较大，因此更适合写成“方向正确但稳定性不足”的结论。运行时字符串搬运批量化方案在 pmc 与 mpegaudio 上只有弱正收益，并在 lu.small 上出现明显回退，因此应被界定为字符串密集场景的定向微优化，而不是通用默认策略。LoongArch64 LSX 浮点 SIMD 向量化方案则在 lu.small 上形成最稳定、最容易解释的收益，是当前最适合作为论文主结论保留的性能优化结果。进一步结合方案叠加负例与失败方案对照，本文归纳出 ART 执行引擎优化中关于热点预算、副作用控制、方案门控、收益稳定性判断和 workload 适用边界的经验，这也是本文区别于单纯展示跑分提升式优化报告的重要之处。

6.2.4 形成以失败样本为约束的优化研究方法

本文没有将失败方案视为应被忽略的噪声，而是将其作为研究结论的一部分系统归档。多项实验表明，局部热点下降并不必然转化为 benchmark 总收益；若改动没有命中主导瓶颈，或者在命中局部热点的同时引入了更大的全局固定

成本，最终结果仍可能明显慢于 baseline。基于这一认识，本文将“固定 baseline、热点定位、最小相关回归、benchmark 复测、失败归档”的闭环方法用于筛选正式方案，从而使论文结论不仅给出“哪些方向有效”，也给出“哪些方向虽然看似合理但在当前口径下并不成立”。这一方法论结论主要由第 5 章中的失败方案热力图、叠加负例与重复测量结果共同支撑。

6.3 工作局限性分析

6.3.1 benchmark 覆盖范围仍有限

本文的正式性能结论建立在五个代表性 workload 之上，即 DaCapo pmd 以及 SPECjvm2008 中的 mpegaudio、lu.small、sor.small 和 fft.small。这些 workload 已覆盖解释器驻留、字符串运行时热路径、短生命周期吞吐和部分规则浮点数值内核等典型行为，但主结论仍主要集中在 pmd 与 lu.small，对并发类 workload、复杂服务型应用和更长生命周期应用的覆盖仍然有限。因此，本文当前结论更适合作为“在固定 JIT 基线下的阶段性优化结果”，而不应直接外推为 LoongArch 平台 ART 的全面性能画像 [26–28]。

6.3.2 部分结论受 debug 构建口径约束

本文的正式优化研究统一建立在 LoongArch64 debug ART 环境下，这一口径有利于在移植尚未完全稳定的阶段保留更多验证信息，但也意味着部分收益结论会受到 debug 构建额外固定成本的影响。特别是 JIT 调试信息相关热点在当前环境中更容易放大，因此本文中关于阈值策略、固定成本和短生命周期 workload 的判断，不宜直接外推到 release 构建、不同系统镜像或其它设备状态下。同时，Java 性能评测中的 warmup 波动与统计不稳定性本身就要求谨慎解释任何单一环境下的结果，这也是本文坚持固定 baseline 和重复复测的原因 [29, 30]。

6.3.3 部分候选方向仍需进一步探索

尽管本文对多类失败方案进行了归纳和分析，但仍有一些围绕解释器快路径、JIT 固定成本削减、体系结构特性利用和更低扰动性能观测的方向值得继续深入。当前论文中的“失败”更多表示在现有平台、现有 build 配置和现有 benchmark 口径下未能形成论文级正收益，而不意味着这些方向在更精细的门控、更广的 workload 组合或更完整的系统配置下永远没有价值。

6.3.4 因果证据仍以外部观测为主

本文已经通过火焰图、符号统计、日志归档和重复 benchmark 复测建立了较完整的外部性能观测链路，但对于 JIT 编译次数、分层切换时机、代码缓存压力以及调试信息生成总量等内部因果指标，当前仍主要依赖间接观测推断。若后续能够补充更低扰动、更细粒度的运行时统计机制，则可进一步增强“热点变化为何转化为最终分数变化”的解释能力。

6.4 后续工作展望

6.4.1 完善 AOT 与安装期编译支持

后续可继续围绕 dex2oat 和安装期编译流程，完善 LoongArch 平台在 AOT 路径上的支持，进一步提升冷启动与安装期执行效率。与本文当前聚焦 JIT 执行阶段不同，AOT 路径能够更直接影响安装期开销、冷启动前若干次执行以及系统预编译策略。未来若能将安装期 profile、JIT 热点迁移和 LoongArch 后端代码质量优化结合起来，ART 在该平台上的整体性能上限仍有继续提升的空间 [5, 6]。

6.4.2 探索 LoongArch 向量扩展优化

后续可在本文已验证的最小 LSX 浮点 SIMD lowering 基础上，继续结合 LoongArch 的 LSX 与 LASX 向量扩展能力，针对更多算子、更多数据类型和更复杂循环模式探索更具体系结构特性的优化手段。当前正式方案只覆盖 32/64 位浮点的基本向量算子，尚未涉及 reduction、除法、条件控制流和更宽向量宽度；因此，更强的体系结构收益仍可能来自字符搬运、编码转换、数组批处理和更完整的浮点向量 lowering。但这一方向同样需要保持谨慎：只有当某类向量化真正命中目标 workload 的主导路径，且不会引入难以接受的代码尺寸、寄存器压力或跨路径副作用时，才适合作为正式默认策略保留 [34, 38]。

本文在正文之外已经完成了一轮“最小 SIMD 版本”与“完整 SIMD 覆盖候选版”的后续消融。结果表明，完整覆盖版虽然在 scimark.lu.small、h2 和 avrora 等关键 workload 上继续取得局部增益，但其全量 JIT 几何平均仍略低于当前正式方案三采用的最小版本。这说明 LoongArch 平台上的 SIMD 后续研究不能简单理解为“支持更多 lowering 就一定更好”，而应进一步回答两个问题：第一，哪些新向量模式值得默认开启；第二，如何把这些模式与更高层的循环识别、idiom

匹配和 pass 级门控结合起来，使新增能力只在真正有利的 workload 上生效。换言之，后续工作的重点已经不只是能否继续补齐 SIMD 能力，而是如何将更完整的 SIMD 能力转化为更稳定的端到端收益。

6.4.3 扩展 benchmark 与应用级验证

后续可在保留现有正式 baseline 方法的基础上，引入更多稳定 benchmark 子项与真实应用场景，包括更丰富的 SPECjvm 数值 workload、DaCapo 其它子项以及更能体现现代 JVM 并发特征的 benchmark 集合。只有将当前“以 pmd 与 lu.small 为主结论、辅以 mpegaudio/sor.small/fft.small 观察边界”的实验口径逐步扩展到更广负载，才能更全面地评估 LoongArch 平台 ART 在不同工作负载下的收益边界，并判断某一优化方向究竟适合默认启用、按 workload 启用，还是只保留为实验性策略 [28]。

6.4.4 增强低扰动观测与运行时因果分析

本文已经初步总结了若干成功与失败方案的经验。后续工作可继续围绕热点预算分析、低副作用局部优化、工作负载分类建模以及低扰动 tracing 机制建设，形成更系统的 LoongArch 平台 ART 执行引擎优化方法论。特别是对于 JIT 分层时机、debug-info 固定成本、代码缓存利用率等问题，若能够在运行时中提供更稳定的统计接口与观测工具，将显著提高后续优化结论的可解释性和迁移性 [31]。

6.5 本章小结

本章对全文工作进行了系统总结，概括了 LoongArch 平台 ART 执行引擎适配、验证方法建立和性能优化研究三个方面的主要成果。本文的结论可以归结为：AOSP 15 ART 在 LoongArch 平台上的执行引擎适配是可行的，但只有在统一验证链路和固定 baseline 的约束下，性能优化结论才具备研究意义；对于当前平台而言，真正值得保留的方案应当同时满足“命中主导热点、作用范围清晰、可通过回归验证、收益边界明确”这四个条件。围绕这些条件，后续仍可在 AOT 路径、向量扩展、benchmark 扩展和低扰动观测等方向继续深入研究。

附录 A 补充实验：DexFileVerifier 阶段链实验

除正文中的正式主线外，本文还围绕 libdexfile/DexFileVerifier 的启动期热点进行过一组独立阶段链实验，用于观察 DEX 校验路径上的局部优化是否能够形成可复现收益。需要强调的是，这组实验采用附录阶段链独立基线版本，与正文第 5 章所使用的正式基线版本以及两项正式优化方案完全隔离，因此本文不将其并入主结论，而是作为补充材料保留。

该阶段链依次包含四组累计改动：第一阶段主要引入文件边界缓存、热点 helper 内联和容器预留容量；第二阶段将 offset_to_type_map_ 的访问模式调整为更紧凑的 vector + lower_bound；第三阶段加入 UTF-8 查表与 ASCII fast path；第四阶段再补充 LEB128 fast path。图 A.1 给出了这四个阶段在 startup.helloworld 与 scimark.lu.small 上的归一化结果。为避免使用难以理解的内部代号，图中横轴采用“阶段号 + 机制名”的标注方式。

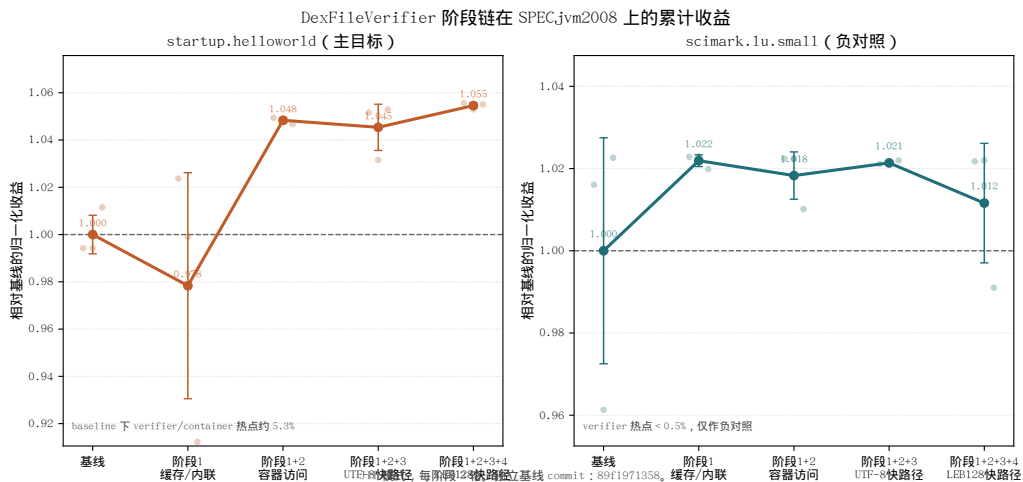


图 A.1 DexFileVerifier 阶段链实验的归一化收益

从结果可以看出，startup.helloworld 是这条阶段链的主要受益对象。第一阶段只完成了局部预清理，尚不足以形成稳定 benchmark 收益；第二阶段首次在 benchmark 层面形成明确正收益；到 phase1234 时，累计收益达到约 +5.46%。与之相对，scimark.lu.small 并未形成稳定提升，这说明该方向更适合作为启动期和加载期路径的定向优化，而不宜被概括为通用 workload 加速方案。

参考文献

- [1] Hu W, Gao J, Wang H. LoongArch: A new RISC ISA [C]//Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA). ACM, 2023.
- [2] 陈华才, 张福新, 武成岗. 龙芯处理器生态建设: 挑战与进展 [J]. 计算机研究与发展, 2024, 61(6): 1280-1295.
- [3] 龙芯中科技股份有限公司. 龙芯架构参考手册卷一: 基础架构 [M]. 2023.
- [4] StatCounter. Mobile operating system market share worldwide [J/OL]. StatCounter Global Stats, 2025. <https://gs.statcounter.com/os-market-share/mobile/worldwide>, 访问于 2026-03-01.
- [5] Google. Android runtime (ART) and dalvik [EB/OL]. 2024. <https://source.android.com/docs/core/runtime>.
- [6] Google. Implementing ART just-in-time (JIT) compiler [EB/OL]. 2024. <https://source.android.com/docs/core/runtime/jit-compiler>.
- [7] Ghuloum A, Carlstrom B, Rogers I. ART: Android's runtime evolved [C]//Google I/O Developer Conference. San Francisco, CA, 2014.
- [8] Bornstein D. Dalvik vm internals [C]//Google I/O Developer Conference. San Francisco, CA, 2008.
- [9] Cheng B, Buzbee B. A jit compiler for android's dalvik vm [C]//Google I/O Developer Conference. San Francisco, CA, 2010.
- [10] Paleczny M, Vick C, Click C. The Java Hotspot server compiler [C]//Proceedings of the Java Virtual Machine Research and Technology Symposium. USENIX Association, 2001: 1-12.
- [11] Suganuma T, Yasue T, Kawahito M, et al. A dynamic optimization framework for a Java just-in-time compiler [C]//Proceedings of the 16th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA). ACM, 2001: 180-195.
- [12] Shi Y, Casey K, Ertl M A, et al. Virtual machine showdown: Stack versus registers [J]. ACM Transactions on Architecture and Code Optimization, 2008, 4(4): 1-36.
- [13] Kotzmann T, Wimmer C, Mössenböck H, et al. Design of the Java HotSpot client compiler for Java 6 [J]. ACM Transactions on Architecture and Code Optimization, 2008, 5(1): 1-32.
- [14] Lee B, Rajan K, Park S. Configuration-aware optimization for Android runtime [C]//Proceedings of the International Conference on Compilers, Architecture, and Synthesis for Embedded Systems (CASES). IEEE, 2020.
- [15] Wang P, Liu Y, Chen H. JavART: Rule-driven JIT compilation for Android application startup

- acceleration [C]//Proceedings of the ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA). ACM, 2022.
- [16] Ertl M A, Gregg D. The structure and performance of efficient interpreters [J]. Journal of Instruction-Level Parallelism, 2003, 5: 1-25.
- [17] Arnold M, Fink S J, Grove D, et al. A survey of adaptive optimization in virtual machines [J]. Proceedings of the IEEE, 2005, 93(2): 449-466.
- [18] Fink S J, Qian F. Design, implementation and evaluation of adaptive recompilation with on-stack replacement [C]//Proceedings of the International Symposium on Code Generation and Optimization (CGO). IEEE, 2003: 241-252.
- [19] Lameed N A, Hendren L J. A modular approach to on-stack replacement in LLVM [C]//Proceedings of the 9th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE). ACM, 2013: 143-154.
- [20] 黄佳. 面向龙芯平台的 Android 运行时环境移植与优化 [D]. 中国科学院大学, 2021.
- [21] 李泽京. 面向 MIPS 平台的 Android ART 运行时技术研究 [D]. 中国科学院大学, 2019.
- [22] Brunthaler S. Efficient interpretation using quickening [C]//Proceedings of the Dynamic Languages Symposium (DLS). ACM, 2010.
- [23] Brunthaler S. Inline caching meets quickening [C]//Proceedings of the 24th European Conference on Object-Oriented Programming (ECOOP). Springer, 2010.
- [24] Pérez G A, Kao C M, Chung Y C, et al. A hybrid just-in-time compiler for Android: Comparing JIT types and the result of cooperation [C]//Proceedings of the International Conference on Compilers, Architecture, and Synthesis for Embedded Systems (CASES). ACM, 2012.
- [25] Häubl C, Wimmer C, Mössenböck H. Optimized strings for the Java HotSpot virtual machine [C]//Proceedings of the 6th International Symposium on Principles and Practice of Programming in Java (PPPJ). ACM, 2008.
- [26] Blackburn S M, Garner R, Hoffmann C, et al. The DaCapo benchmarks: Java benchmarking development and analysis [C]//Proceedings of the 21st ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA). ACM, 2006: 169-190.
- [27] Standard Performance Evaluation Corporation. SPECjvm2008 benchmarks [EB/OL]. 2008. <https://www.spec.org/jvm2008/>.
- [28] Prokopec A, Rosà A, Leopoldseder D, et al. Renaissance: Benchmarking suite for parallel applications on the JVM [C]//Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). ACM, 2019.
- [29] Georges A, Buytaert D, Eeckhout L. Statistically rigorous Java performance evaluation [C]//

- Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA). ACM, 2007.
- [30] Barrett E, Bolz-Tereick C F, Killick R, et al. Virtual machine warmup blows hot and cold [J]. arXiv preprint arXiv:1602.00602, 2017.
- [31] Luo Y, Rodrigues K, Li C, et al. Hubble: Performance debugging with in-production, just-in-time method tracing on Android [C]//Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2022.
- [32] Wimmer C, Würthinger T, Mössenböck H. Cross-platform code generation with Graal [C]//Proceedings of the ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages (VMIL). ACM, 2013.
- [33] Loongson Technology Corporation Limited. Loongarch ELF ABI specification [M/OL]. 2025. <https://loongson.github.io/LoongArch-Documentation/LoongArch-ELF-ABI-EN.html>.
- [34] Loongson Technology Corporation Limited. Toolchain conventions of the loongarch architecture [M/OL]. 2025. <https://github.com/loongson/la-toolchain-conventions>.
- [35] 龙芯中科. OpenJDK LoongArch 移植 [EB/OL]. 2023. <https://github.com/ABI-of-LoongArch>.
- [36] Aho A V, Lam M S, Sethi R, et al. Compilers: Principles, techniques, and tools [M]. 2nd ed. Addison-Wesley, 2006.
- [37] Xu X, Cooper K, Brock J, et al. ShareJIT: JIT code cache sharing across processes and its practical implementation [C]//Proceedings of the ACM on Programming Languages. 2018.
- [38] 龙芯中科技术股份有限公司. 龙芯架构参考手册卷二：向量扩展 [M]. 2023.

致 谢

本论文是在导师张福新老师的指导下完成的。从课题选题、研究路线梳理、平台适配推进，到实验设计、结果分析与论文撰写，张老师都给予了我持续而细致的指导。导师严谨务实的治学态度、对工程细节的高标准要求以及对研究问题边界的反复推敲，使我在整个硕士阶段受益良多。在此谨向张老师致以最诚挚的感谢。

感谢中国科学院计算技术研究所提供的学习与科研环境，感谢课题组老师和同学们在 ART 运行时、LoongArch 平台适配、实验环境维护和问题讨论中给予的帮助。论文研究过程中，无论是目标机调试、benchmark 复测，还是失败方案复盘，组内交流都为我提供了许多直接而有效的启发。

感谢课题相关前期工作的积累以及开源社区提供的基础软件与文档支持，使本文能够在已有工程基础上继续推进 AOSP 15 ART 在 LoongArch 平台上的适配与优化研究。对所有在平台搭建、工具维护和资料整理中给予帮助的老师 and 同学，在此一并致谢。

最后，感谢家人长期以来的理解、支持与陪伴。在论文撰写与实验推进过程中，正是他们的包容和鼓励，让我能够持续投入并完成本阶段的学习与研究工作。

作者简历及攻读学位期间发表的学术论文与研究成果

作者简历：

鄢玺，中国科学院计算技术研究所硕士研究生，研究方向为 Android 运行时系统移植与性能优化。

已发表（或正式接受）的学术论文：

攻读硕士学位期间暂无公开发表或正式接受的学术论文。

申请或已获得的专利：

攻读硕士学位期间暂无申请或已获得的专利。

参加的研究项目及获奖情况：

1. 参与基于 LoongArch 架构的 Android 运行时适配与执行引擎优化相关研究，完成 AOSP 15 ART 执行引擎适配、验证链路建立以及 benchmark 驱动的性能分析与优化工作。

攻读硕士学位期间暂无单独列出的获奖情况。

